



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE  
ESCUELA DE INGENIERÍA  
ICS2121 – MÉTODOS DE OPTIMIZACIÓN

---

# Apuntes de Métodos de Optimización

---

*Por Francisco García Aubert y Jorge Vera Andreo  
Material en desarrollo. No compartir fuera del curso.  
(versión 7, Agosto 2025)*



# Índice general

|                                                                             |          |
|-----------------------------------------------------------------------------|----------|
| <b>2. Métodos de Primer Orden</b>                                           | <b>3</b> |
| 2.1. Problemas diferenciables . . . . .                                     | 4        |
| 2.1.1. Gradiente sin Linesearch . . . . .                                   | 4        |
| 2.1.2. Convergencia Método Simple . . . . .                                 | 8        |
| 2.1.3. Propiedad fuerte de convexidad . . . . .                             | 8        |
| 2.1.4. Método acelerado de Nesterov . . . . .                               | 9        |
| 2.2. Problemas no diferenciables . . . . .                                  | 11       |
| 2.2.1. LASSO: Least Absolute Shrinkage and Selection Operator . . . . .     | 11       |
| 2.2.2. El Método del Subgradiente . . . . .                                 | 13       |
| 2.2.3. FISTA: Fast Iterative Shrinkage and Thresholding Algorithm . . . . . | 17       |
| 2.2.4. Resumen . . . . .                                                    | 20       |
| 2.3. Métodos de Primer Orden con Restricciones . . . . .                    | 20       |
| 2.4. Ejemplos . . . . .                                                     | 24       |
| 2.4.1. Función no diferenciable restringida . . . . .                       | 24       |
| 2.4.2. Subgradiente de la función “Max” . . . . .                           | 26       |
| 2.4.3. Subgradiente de función “max” combinadas con otras . . . . .         | 27       |
| 2.4.4. Subgradiente y esfuerzo computacional . . . . .                      | 28       |
| 2.4.5. Subgradiente y esfuerzo computacional 2 . . . . .                    | 30       |
| 2.4.6. Método de Primer Orden proyectado . . . . .                          | 31       |
| 2.4.7. Método no diferenciable proyectado . . . . .                         | 32       |

## Capítulo 3

# Métodos de Primer Orden

En muchos problemas estadísticos, estimar una cierta variable en función de otras lleva a querer resolver un sistema de ecuaciones de la forma

$$Ax = b$$

Donde  $A \in \mathbb{R}^{m \times n}$ , y  $b \in \mathbb{R}^m$ . En los últimos tiempos han surgido problemas de gran tamaño, con muchísimas variables. Estos problemas surgen en diversas áreas de aplicación: en econometría, en reconstrucción de imágenes médicas, en problemas de “Machine Learning” en Internet, es decir en todo lo que se conoce como *Big Data*. En muchos de estos problemas se tiene que  $m \ll n$ , por lo que el sistema  $Ax = b$  queda subdeterminado y tiene infinitas soluciones posibles. Un enfoque, que estudiaremos en detalle más adelante, es buscar una solución que tengas pocas componentes no nulas, es decir, pocos factores explicativos. Para lograr pocas variables distintas de cero ( $x_i \neq 0$ ), el problema original se suele resolver mediante mínimos cuadrados, pero agregando lo que se conoce como una “Regularización L-1”:

$$\min_{x \in \mathbb{R}^n} f(x) = \tau \|x\|_1 + \|Ax - b\|_2^2$$

donde  $\tau > 0$  es un parámetro dado. El segundo término minimiza el error cuadrático de ajuste de  $x$  (el cual es convexo y diferenciable), mientras que el primer término busca “bajar” la densidad del vector  $x$  (más adelante explicaremos en detalle la lógica de esto). El factor  $\tau$  se usa para manejar el balance entre el ajuste al sistema de ecuaciones y la densidad del vector  $x$ . De este modo, tenemos un problema con una función objetivo  $f(x)$  es continua, convexa pero que no es diferenciable.

Lo anterior genera de inmediato el problema de que nuestros métodos del capítulo anterior, el Método de Gradiente y el Método de Newton, no podrían usarse ya que requieren derivadas.

Por otro lado, es posible reemplazar  $\|x\|_1$  por una función aproximada, llamémosla  $g(x)$ , de modo tal que si consideramos el problema

$$\min_{x \in \mathbb{R}^n} f(x) = \tau g(x) + \|Ax - b\|_2^2$$

este sí será diferenciable y podremos usar los métodos de Newton y Gradiente.

Bueno, no es tan simple. Utilizar Newton para este tipo de problemas no servirá en la práctica ya que las aplicaciones en que esto se usa llevan a problemas con cientos de miles de variables, en algunos casos millones. Esto hará que Newton sea extremadamente lento debido a su costo  $\mathcal{O}(n^3)$  por iteración. Es por esto que en este tipo de problemas tan grande uno quisiera utilizar el método del gradiente que es  $\mathcal{O}(n)$  o algo del estilo del método del Gradiente que sea rápido para problemas grandes. Notemos aquí que, incluso las iteraciones que se hagan para resolver el problema de “linesearch” pueden agregar costos importantes. Por eso razón, queremos tener un Método de Gradiente que sea lo más económico posible por iteración.

En este capítulo se aborda la forma en que métodos basados en gradientes (o sus generalizaciones, para el caso no diferenciable) pueden usarse para abordar eficientemente toda esa categoría de problemas, especialmente

en el contexto de grandes dimensiones. Todo este tema ha sido uno de los desarrollos importantes en Optimización durante el Siglo XXI.

Específicamente, comenzaremos con problema diferenciables pero en los cuales la cantidad de variables es muy grande. Veremos, entonces, cómo obtener un Método de Gradiente que sea realmente eficiente. Posteriormente, revisaremos qué hacer en el caso de funciones no diferenciables, como es originalmente el problema de estimación estadística que planteamos. Esto nos llevará a extender la noción de “gradiente” mediante el estudio de algunos elementos básicos de optimización no diferenciable.

### 3.1. Problemas diferenciables

Tal como explicamos, comenzaremos por estudiar el problema diferenciable y mostraremos como el tradicional Método del Gradiente puede ser modificado para que tenga un bajo costo por iteración.

#### 3.1.1. Gradiente sin Linesearch

El problema de optimización es el mismo de antes:

$$P) \min_{x \in \mathbb{R}^n} f(x)$$

Donde  $f : \mathbb{R}^n \rightarrow \mathbb{R}$ . Esta vez, agregamos la hipótesis de que  $f$  es **convexa**, continua y diferenciable. Uno de los problemas del Método del Gradiente, que es de primer orden, consiste en que en cada una de las iteraciones tiene que calcular el paso de avance  $\lambda^k$  resolviendo un subproblema de optimización, cosa que aunque no se haga de modo exacto en muchas ocasiones, implica costos de tiempo adicional. Una forma de solucionar esto es no resolver el *linesearch*. Esto se puede hacer haciendo que  $\lambda^k$  esté definido a priori, de modo que no se requieran cálculos iterativos de ningún tipo. Esto es usar el Método del Gradiente, pero sin *linesearch*. Veamos cómo es posible lograr eso.

Como es sabido, si  $f$  es convexa, entonces el hiperplano tangente siempre estará bajo el gráfico de la función como una cota inferior, para un  $x$  fijo:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) = l_x(y), \quad \forall x, y \in \mathbb{R}^n$$

Esto nos da una aproximación lineal por debajo ( $l_x(y)$ ) de  $f(y)$ . Agregaremos, adicionalmente, el supuesto de que  $\nabla f(x)$  es L-Lipschitziana, con parámetro  $L > 0$ :

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|, \quad \forall x, y \in \mathbb{R}^n,$$

de modo que la variación *de los gradientes* está acotada por la variación de los puntos donde estamos evaluando. Notar que, por el Teorema del Valor Medio, lo que debería acotar a la variación de los gradientes entre dos puntos  $x$  e  $y$  es el Hessiano. En este caso, en vez del Hessiano se utiliza  $L$ , que hace un papel análogo pero que no requiere ninguna matriz de gran tamaño.

Entonces, para un determinado valor de  $L$ , se puede probar que  $f$  tiene una cota superior, para un  $x$  fijo:

$$f(y) \leq f(x) + \nabla f(x)^T(y - x) + \frac{L}{2}\|y - x\|_2^2 = q_x(y), \quad \forall x, y \in \mathbb{R}^n$$

Acá no hay ninguna segunda derivada que evaluar, ningún Hessiano. Las dos expresiones mencionadas nos dan una cota inferior y una superior para  $f$ :  $l_x(y) \leq f(y) \leq q_x(y)$ ,  $y \in \mathbb{R}^n$ .

Ahora nos interesa el valor de  $y$  que minimiza la aproximación superior  $q_x(y)$ , es decir, queremos resolver el siguiente problema:

$$\min q_x(y) = \min \left\{ f(x) + \nabla f(x)^T(y - x) + \frac{L}{2}\|y - x\|_2^2 \right\}$$

Este mínimo se encuentra derivando e igualando a cero:

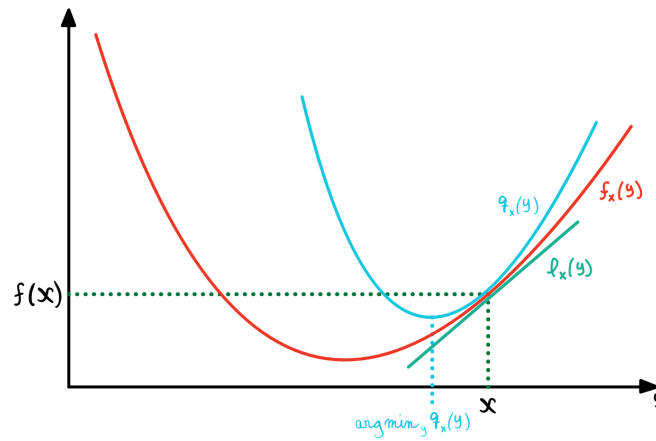


Figura 3.1: Cotas inferior lineal y superior cuadrática

$$\begin{aligned}\nabla q_x(y) = 0 &\Leftrightarrow \nabla f(x) + L(y - x) = 0 \\ y &= x - \frac{1}{L} \nabla f(x)\end{aligned}$$

Esta expresión es similar a la del Método del Gradiente pero con paso  $1/L$ . Se puede demostrar que este paso:

$$\lambda^k = \frac{1}{L}, \quad \forall k$$

satisface la condición de Wolfe-Armijo. El algoritmo del Método del Gradiente equipado con este nuevo paso es más barato que Newton ya que requiere calcular únicamente el gradiente y más barato que el Método del Gradiente clásico ya que no requiere iteraciones adicionales para resolver un problema de Linesearch.

#### Cápsula de video 3.1.1: Deducción del paso para el Método Simple de Primero Orden

Profesor Jorge Vera explica en clases la deducción del paso fijo a partir de las propiedades de convexidad y “lipschitzianidad” del problema:  
<https://youtu.be/osa-2CoAxfU>

El algoritmo se muestra a continuación:

#### Método simple de Primer Orden (Gradiente sin Linesearch)

**Data:** Sea  $x^0 \in \mathbb{R}^n$ ,  $k = 0$  y  $\epsilon > 0$   
**while**  $\|d^k\| > \epsilon$  **do:**  
 $x^{k+1} = x^k + \frac{1}{L} d^k$   
 $d^{k+1} = -\nabla f(x^{k+1})$ .  
 $k = k + 1$   
**end**

En análisis de esta idea es antiguo, fue desarrollado por Arkadi Nemirovsky en los 70's y completado por Yuri Nesterov en los 80's, más otra cantidad de desarrollo de muchos otros investigadores. Para implementar este algoritmo falta conocer el valor de  $L$ , pero según el problema, esto se puede hacer directamente o simplemente aproximar. Más aún, lo que realmente importa es que la función tenga este comportamiento en la región del espacio que es de interés. Si bien puede no ser claro cuál es esa región (ya que necesitaríamos conocer el mínimo para determinarla), en muchos problemas se sabe, por estructura, cuáles son los valores posibles. Lo fundamental, entonces, es que se cumpla con las dos condiciones de las que se deduce este método:  $l_x(y) \leq f(y) \leq q_x(y)$ . En primer lugar, recordemos que;

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|, \quad \forall x, y \in \mathbb{R}^n$$

Reordenando estos términos se tiene:

$$\frac{\|\nabla f(x) - \nabla f(y)\|}{\|x - y\|} \leq L, \quad \forall x, y \in \mathbb{R}^n$$

Si se toman valores de  $x$  y de  $y$  que estén muy cerca, entonces el lado derecho de la ecuación se parece a algo como la norma de una segunda derivada de  $f$  en la dirección  $x - y$ , de modo que tiene que ver con la norma del hessiano de  $f$ . Así, el valor de  $L$  se puede calcular como el mayor valor propio de la matriz de segundas derivadas de  $f$ , sobre todos los posibles puntos  $x$ :

$$L = \max_x \{\|\nabla^2 f(x)\|_2\}$$

De este modo, se puede utilizar como valor de  $L$  la norma de  $\nabla^2 f(x)$  que es igual al mayor valor propio:  $L = \|\nabla^2 f(x)\|_2 = \text{máx valor propio}$ . En un algoritmo que aplique esto, podría demorarse únicamente cuando se inicializa y calcula la norma de  $\nabla^2 f(x)$ , pero después de eso, es rápido. En un algoritmo de este tipo, las direcciones consecutivas de dos vectores de descenso son casi colineales, por lo que se mueve de manera muy distinta al “zigzag” ortogonal del Método del Gradiente.

Con el paso  $1/L$ , el tiempo por iteración es menor que con el método del gradiente tradicional, que debe calcular  $\lambda$ , mientras que teniendo el paso predeterminado implica que no se debe hacer ningún cálculo. Sin embargo, como veremos más adelante, habrá que pagar un precio por esto y eso se traduce en que se deben realizar más iteraciones.

### Ejemplo: Mínimos Cuadrados

El problema de estimación estadística mencionado al principio requiere resolver el sistema:

$$Ax = b$$

Un enfoque de solución a este problema puede ser el de mínimos cuadrados:

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} \|Ax - b\|_2^2$$

Para resolver este problema mediante el Método del Gradiente sin *linesearch* se debe calcular el gradiente de la función y un valor apropiado de  $L$ . La función  $f$  se puede trabajar del siguiente modo:

$$\begin{aligned} \frac{1}{2} \|Ax - b\|_2^2 &= \frac{1}{2} (Ax - b)^T (Ax - b) \\ &= \frac{1}{2} x^T (A^T A) x - b^T A x + \frac{1}{2} b^T b \end{aligned}$$

Sea  $Q = A^T A$ ,  $c = -A^T b$  y  $\alpha = \frac{1}{2} b^T b$ , entonces se puede reescribir  $f(x)$  como:

$$f(x) = \frac{1}{2} x^T Q x + c^T x + \alpha$$

De este modo,  $\nabla f(x) = Qx + c$  y a su vez  $\nabla^2 f(x) = Q$ , por lo que se puede concluir que un valor de  $L$ , que es la constante de Lipschitz de los gradientes, se puede calcular como  $L = \|\nabla^2 f(x)\|_2$ , por lo que  $L$  es el mayor de los valores propios de  $Q$ :  $L = \|Q\|_2 = \|A^T A\|_2$  (recordar que la norma dos de las matrices corresponde al mayor valor propio). Esto se puede calcular con facilidad en un computador, por ejemplo, en *Matlab*, se puede hacer con el comando `norm(Q)`.

Computacionalmente, si se evalúa el producto punto de la dirección entre dos iteraciones seguidas, suele suceder que dicho valor se encuentre entre 0.9 y 1.0, lo que significa que se mueve prácticamente en línea recta, con una ligera curva para llegar al óptimo. Eso significa que se aproxima de modo más directo que como se mueve el “zigzag” Método del Gradiente con *linesearch*.

Sin embargo, en cada paso, el algoritmo avanza muy poco, por lo que debe hacer muchas más iteraciones. Cabe recordar, eso sí, que lo que finalmente nos importa es el tiempo total que tarda el algoritmo en llegar a la meta y ese tiempo depende no solo del número de iteraciones sino también del tiempo invertido en cada iteración. Un algoritmo puede hacer pocas iteraciones pero si cada una de ellas es muy larga, tardará mucho. La promesa de lo que estamos desarrollando aquí es que, aún con muchas iteraciones, el bajo costo en tiempo de cada una de ellas permita llegar a la meta antes.

A continuación se muestra un código Python para abordar el problema descrito mediante el algoritmo.

```
import numpy as np
from apendice import generar_datos, angulo_entre_dos_iteraciones

def fomq(A, b, x0, epsilon, iteracion_maxima):
    """
    Funcion que aplica el metodo simple de primer orden a una funcion cuadratica
    definida por Q y c para resolver de manera aproximada el problema: Ax = b.
    Los argumentos de entrada son:
        - La matriz A y el vector b. del problema en cuestion
        - x0: punto inicial.
        - epsilon: tolerancia deseada.
        - iteracion_maxima: numero maximo de iteraciones permitido.
    Su retorno es:
        - val: valor alcanzado.
        - x: punto donde se alcanza ese valor.
    """
    k = 0
    Q = np.matmul(np.transpose(A), A)
    c = -np.dot(np.transpose(A), b)
    # x es el punto actual x_1 es x(k-1) y x_2 es x(k-2)
    # Se usan para evaluar el avance de la trayectoria del algoritmo
    x = x0
    x_1 = x_2 = x
    grad = np.dot(Q, x) + c
    L = np.linalg.norm(Q)

    while k < iteracion_maxima:
        # Aca se hacen los calculos fundamentales del algoritmo
        h = - grad
        x = x + (1 / L) * h
        val = (np.dot(np.transpose(x), np.dot(Q, x)) / 2 +
              np.dot(np.transpose(c), x)) [0] [0]
        grad = np.dot(Q, x) + c
        if np.linalg.norm(grad, 2) < epsilon: break
        # Se calcula el producto interno entre los vectores de avance
        # en iteraciones sucesivas (Esta linea no es necesaria)
        x_1, x_2 = angulo_entre_dos_iteraciones(k, x, x_1, x_2)
        k = k + 1

    return val, x

if __name__ == '__main__':
    np.random.seed(2121)
    n = 30
    A, b = generar_datos(n)
    epsilon, iteracion_maxima, x0 = 0.0001, 2000000, np.ones((n, 1))

    val, x = fomq(A, b, x0, epsilon, iteracion_maxima)
```

**Cápsula de video 3.1.2: Problema de Mínimos Cuadrados**

Profesor Jorge Vera desarrolla la preparación del problema de Mínimos Cuadrados lineales para ser abordado mediante un método de primer orden simple.

<https://youtu.be/5tdIzzq1nTQ>

**3.1.2. Convergencia Método Simple**

Sea  $f$  convexa y continuamente diferenciable y supongamos que  $\nabla f$  es  $L$ -Lipschitz. Entonces, se puede demostrar que después de  $k$  iteraciones, se tiene que:

$$f(x^k) - f(x^*) \leq \frac{L \|x^0 - x^*\|_2^2}{2k}$$

Si se desarrolla esta expresión para encontrar la cantidad de iteraciones que se deben realizar para tener una solución con error garantizadamente menor que  $\epsilon > 0$ , basta imponer que el lado derecho de la desigualdad sea  $\leq \epsilon$ :

$$\begin{aligned} \frac{L \|x^0 - x^*\|_2^2}{2k} &\leq \epsilon \\ \frac{L \|x^0 - x^*\|_2^2}{2\epsilon} &\leq k \end{aligned}$$

Luego, la cantidad de iteraciones que se deben realizar es del orden de  $\mathcal{O}(1/\epsilon)$ , por lo que la convergencia es mucho peor que la convergencia lineal  $\mathcal{O}(\log(1/\epsilon))$  que presenta el Método del Gradiente clásico. De todos modos, como indicábamos, debido a lo barato que es por iteración, el tiempo final de ejecución del algoritmo para problemas muy grandes puede ser mucho menor que el del Método del Gradiente. Más aún, en muchos problemas, tal como es el entrenar redes neuronales, no se exige un  $\epsilon$  tan pequeño (e.g.  $10^{-2}$  basta). Por otro lado, existe la posibilidad de acelerar el algoritmo con el método acelerado de Y. Nesterov, que tiene complejidad de  $\mathcal{O}(1/\sqrt{\epsilon})$ , que utiliza dos sucesiones de puntos que van “intercambiando información” de cómo hay que moverse, de modo que se mueven marginalmente más rápido. Esto es lo que se implementa en los métodos verdaderos. Esto se explica en la sub sección “Método acelerado” más adelante.

**3.1.3. Propiedad fuerte de convexidad**

Hasta ahora a  $f$  solo le hemos pedido ser convexa y que  $\nabla f(x)$  tenga la propiedad de Lipschitz. Si es que además se cumple que  $f$  tiene propiedades fuertes de convexidad ( $\nabla^2 f(x)$  definida positiva para todo  $x$ , y por tanto todos sus valores propios son estrictamente mayores a 0), entonces el método con el paso de  $1/L$  tiene convergencia lineal. Esto sucede debido a que no se “aplana” la función en ningún  $x$ , por lo que a los algoritmos “les cuesta menos elegir la dirección de descenso”. La definición formal es la siguiente:

**Definición 3.1.1: Convexidad Fuerte**

Sea  $f$  convexa, con  $\nabla f$   $L$ -Lipschitz, se dice que  $f$  es fuertemente convexa si existe  $\mu > 0$ , tal que:

$$f(y) \geq f(x) + \nabla f(x)^T(y - x) + \frac{\mu}{2} \|y - x\|_2^2, \quad \forall x, y \in \mathbb{R}^n$$

Notar que la función cuadrática del lado derecho es una cota inferior para  $f$  que viene dado por la convexidad fuerte. Además ya se había encontrado una cota superior, también cuadrática, que venía por la propiedad Lipschitziana de  $\nabla f$ . De este modo,  $f$  se encuentra acotada por dos funciones cuadráticas:

$$f(x) + \nabla f(x)^T(y - x) + \frac{\mu}{2} \|y - x\|_2^2 \leq f(y) \leq f(x) + \nabla f(x)^T(y - x) + \frac{L}{2} \|y - x\|_2^2, \quad \forall x, y \in \mathbb{R}^n$$

Bajo la hipótesis anterior, existe el siguiente Teorema de Convergencia para el Método Simple:

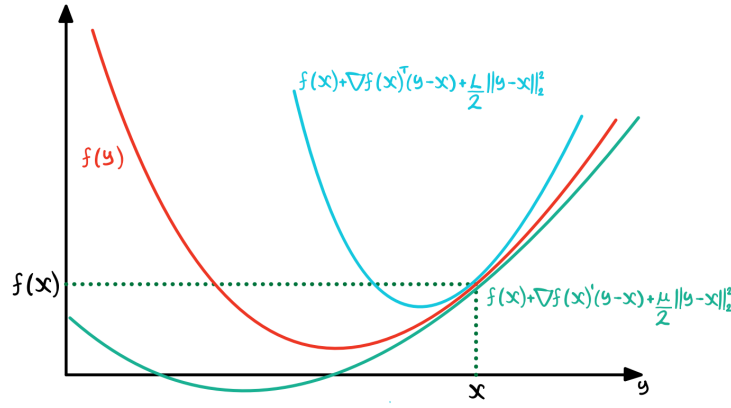


Figura 3.2: Cotas inferior y superior cuadráticas

**Teorema 3.1.1: Convergencia bajo Convexidad Fuerte**

Si  $f$  es fuertemente convexa de parámetro  $\mu$  y  $\nabla f$  es Lipschitziana de parámetro  $L$ , entonces para todo  $k$  se tiene:

$$\|x^{k+1} - x^*\| \leq \left( \frac{\kappa_f - 1}{\kappa_f + 1} \right)^k \|x^k - x^*\|$$

Donde  $\kappa_f = L/\mu$ .

Además, se puede usar que  $L = \lambda_{\max}$  y  $\mu = \lambda_{\min}$ , de modo que queda similar al número de condicionamiento de la matriz  $Q$ ,  $\kappa(Q) = \lambda_{\max}/\lambda_{\min}$  que se utilizaba en el método del Gradiente aplicado a un problema cuadrático. Utilizando todo esto, la convergencia es lineal, es decir, para llegar a una solución  $\epsilon$ -aproximada basta realizar una cantidad de iteraciones del orden:

$$\mathcal{O}\left(\log\left(\frac{1}{\epsilon}\right)\right)$$

Esto corresponde a una velocidad lineal. Nemirovski y Yudin demostraron que ningún método que este inspirado en direcciones de gradiente puede ser más rápido que esto. El argumento de ellos no lo mostraremos aquí dado que es extremadamente técnico, pero se basa en lo que se conoce como un “argumento adversario” o un “oráculo resistivo”: esto consiste en asumir que uno podría tener un algoritmo que logre un número de iteraciones menor y mostrar que siempre existirá un caso de función que no funciona para ese algoritmo y requiere, al menos, el número de iteraciones indicadas por el teorema.

**3.1.4. Método acelerado de Nesterov**

Por lo visto hasta ahora, tenemos que, salvo en los casos que se impongan hipótesis adicionales (como convexidad fuerte), los métodos simples basados en gradientes pueden ser muy lentos, llegando a ser hasta de  $\mathcal{O}(1/\epsilon)$ , lo que es muy lento cuando el error,  $\epsilon$ , que se busca es pequeño. Yuri Nesterov en los años 80s desarrolló ideas para acelerar el método del gradiente, lo que permite que las iteraciones avancen más rápido.

Este método explora el espacio dinámicamente, utilizando dos sucesiones de puntos en vez de una. Sea un  $x^0 \in \mathbb{R}^n$ , y sea  $z^0 = x^0$ . En la iteración  $k$  con  $x^k$  y  $z^k$ , calculamos:

$$\begin{aligned} z^{k+1} &\leftarrow z^k - \frac{1}{\theta_k L} \nabla f(\theta_k z^k + (1 - \theta_k) x^k) \\ x^{k+1} &\leftarrow (1 - \theta_k) x^k + \theta_k z^{k+1}, \end{aligned}$$

donde  $\theta_k$  es una sucesión que converge a cero y  $L$  es la constante de Lipschitz de  $\nabla f$ . En el fondo,  $z^{k+1}$  se mueve en la dirección del negativo del gradiente, pero este es evaluado en un punto intermedio entre  $x^k$  y  $z^k$ , con un paso de  $1/\theta_k L$ . El nuevo  $x$  se construye entre  $x^k$  y  $z^{k+1}$ . La interpretación es que así  $z^k$  “arrastra” a  $x^k$  a distancias que van creciendo debido al hecho que  $\theta_k$  va disminuyendo (ver Figura 2.3).

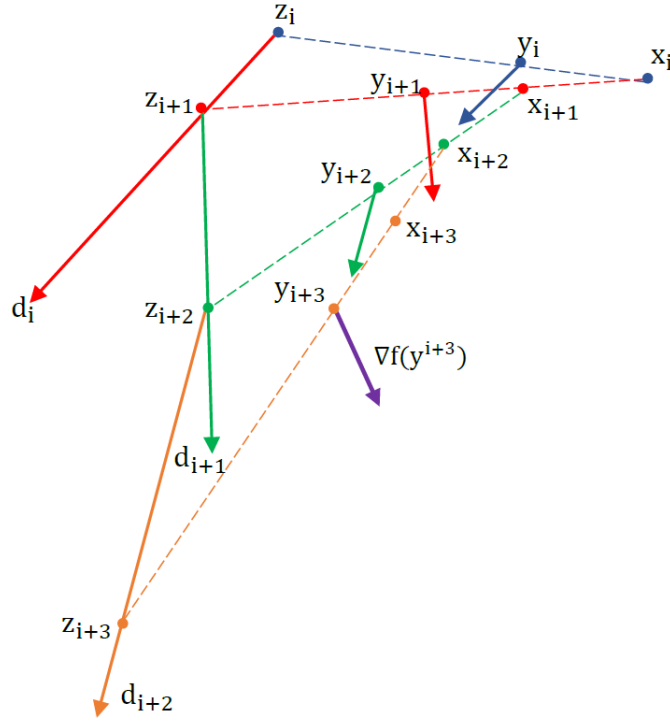


Figura 3.3: Tres Iteraciones con el método Acelerado de Nesterov

Este método logra, efectivamente, ser más eficiente que el método básico de primer orden. Una versión del teorema de convergencia desarrollado por Nesterov para medir el error de la solución de la iteración  $k$  es la siguiente:

**Teorema 3.1.2: Convergencia del Método Acelerado de Primer Orden**

Sea  $f$  convexa y  $\nabla f$   $L$ -Lipschitz, con  $\theta_k = \frac{2}{2+k}$ . Después de  $k$  iteraciones se tiene:

$$\min_{0 \leq i \leq k} f(x^i) \leq f(x^*) + \frac{2L \|x^0 - x^*\|_2}{(k+1)^2}$$

Nótese que el término que está a la izquierda no es  $f(x^k)$ , sino que es el mejor de todos los valores que se han encontrado hasta la iteración  $k$ . Esto se debe a que no necesariamente la mejora de la función objetivo es monótona, como sí ocurre en el método del gradiente (ya que la dirección de descenso siempre es el gradiente del punto  $x^k$ ). En este caso, como no se toma la dirección de descenso exactamente en el punto  $x^k$ , puede haber oscilaciones locales, pero en el global el método va disminuyendo el valor de la función objetivo. Se puede apreciar que el segundo término del lado derecho corresponde al error, por lo que si queremos ejecutar el algoritmo hasta alcanzar un error inferior a  $\epsilon$ , entonces el número de iteraciones necesarios tiene por cota inferior:

$$\frac{2L \|x^0 - x^*\|_2}{(k+1)^2} \leq \epsilon$$

$$k \geq \sqrt{\frac{2L \|x^0 - x^*\|_2}{\epsilon}} - 1,$$

por lo que la complejidad del algoritmo es del orden de  $\mathcal{O}(1/\sqrt{\epsilon})$ , lo que es mejor a  $\mathcal{O}(1/\epsilon)$  del método sin acelerar. Esta es la gran ventaja del método acelerado (de ahí su nombre) y puede, efectivamente, ser muy eficiente en aplicaciones prácticas. El esfuerzo computacional por iteración no es significativamente diferente al del método del gradiente básico y también requiere la evaluación del gradiente en un punto.

Comentemos, adicionalmente, que si hay convexidad fuerte y condición de  $L$ -Lipschitz de  $\nabla f(x)$ , entonces este método también adquiere convergencia lineal, es decir, llega a ser de orden de  $\mathcal{O}(\log(1/\epsilon))$ .

## 3.2. Problemas no diferenciables

Como decíamos en la introducción a este capítulo, Un conjunto importante de problema aplicados, especialmente algunos que aparecen en estimaciones estadísticas, reconstrucción de imágenes y otras, presentan muchas variables pero con una función objetivo que no es necesariamente diferenciable en todas parte. En esta sección mostramos cómo abordar estos problemas. Partiremos por explicar en más detalle el problema estadístico que mostrábamos al comienzo.

### 3.2.1. LASSO: Least Absolute Shrinkage and Selection Operator

En muchas aplicaciones de reconstrucción de imágenes (*Compressed Sensing*), reconstrucción de información (Sistemas recomendadores y Componentes Principales), *Machine Learning* y *Big Data*, surgen problemas con muchísimas variables ( $n$  grande) e información muy dispersa. En estos problemas se deben resolver sistemas de ecuaciones lineales  $Ax = b$ , con  $x \in \mathbb{R}^n$ ,  $A \in \mathbb{R}^{m \times n}$  y  $b \in \mathbb{R}^m$ , pero se tiene que  $m \ll n$ , por lo que el sistema está subdeterminado (tiene muchas más variables que restricciones). Lo que se busca, entonces, es una solución  $x$ , que sea lo más “rala” posible (“sparse”, se dice en Inglés). Esto significa imponer que pocas componentes de  $x$  sean distinto de cero ( $x_i \neq 0$ ). Esto equivale a tratar de identificar el menor número posible de factores explicativos.

#### Ejemplo: Estudio de enfermedades en función de la genética

Ejemplo de lo anterior es la investigación de una enfermedad en función de las características genéticas de los pacientes. La enfermedad puede presentar distintos niveles de gravedad y se plantea la hipótesis de que esos niveles de gravedad podrían ser explicados por el grado de expresión de ciertos genes (la expresión de un gen es la forma en que su codificación se manifiesta en el organismo). El problema en esto es que los genes,  $n$  en total, que se estudiarían pueden ser miles, mientras que la cantidad de pacientes  $m$  que presentan la enfermedad que se pueden estudiar son considerablemente menos  $m \ll n$ . Supongamos que tenemos como datos el grado de severidad del paciente  $i$ ,  $y_i$ , y también tenemos la información del grado de expresión del gen  $j$  en el paciente  $i$ ,  $a_{ij}$ . Entonces, interesa estudiar qué genes explican la enfermedad, para lo que se plantea un modelo lineal, donde  $x_j$  es el peso del gen  $j$ :

$$y_i = \sum_{j=1}^n a_{ij}x_j + e_i$$

donde, además,  $e_i$  es un término de error estadístico de la estimación. Este es un problema de regresión lineal de la forma  $y = Ax + e$ . Los problemas de regresión lineal se abordan tratando de encontrar  $x$  de modo que  $Ax$  sea lo más parecido a  $y$ , lo que se hace, habitualmente, mediante mínimos cuadrados que, en este caso, llevaría a un problema del tipo

$$\min_x \sum_i e_i^2 = \min_x \sum_{i=1}^m (y_i - \sum_{j=1}^n a_{ij}x_j)^2$$

Ahora, dado que el problema está sobredeterminado, este problema de optimización tiene múltiples soluciones y todas dan un óptimo igual a 0, lo que es totalmente inútil. De ahí surge, entonces, la idea de buscar aquella solución específica tal que pocos genes sean realmente explicativos de la enfermedad. Esto se puede plantear como un problema en que se quiere minimizar la cantidad de componentes  $x_j \neq 0$ . Esto lleva al siguiente problema:

$$\begin{aligned} \min \quad & \text{card}\{j : x_j \neq 0\} = \|x\|_0 \\ \text{s.a.} \quad & Ax = y \end{aligned}$$

Este problema es de naturaleza combinatorial, y se puede modelar mediante programación entera, definiendo las siguiente variables:  $z_j = \{1 \text{ si } x_j \neq 0, 0 \text{ e.o.c.}\}$  Si se conoce  $M$  tal que es mayor que el módulo de cualquier componente de  $x$  ( $|x_j| \leq M$ ), entonces, el problema de optimización queda definido del siguiente modo:

$$\begin{aligned} \min \quad & \sum_{j=1}^n z_j \\ \text{s.a.} \quad & Ax = y \\ & -Mz_j \leq x_j \leq Mz_j, \quad \forall j \\ & z_j \in \{0, 1\}, \quad \forall j \end{aligned}$$

Este problema de programación entera, al ser de miles de variables resulta, en general, intratable de resolver en el computador, por lo que conviene utilizar un enfoque alternativo. Lo que hay que ver acá es que en  $Ax = y$ , la matriz  $A$  es “alargada para el lado” pues  $m \ll n$ , entonces, si pensáramos en soluciones “básicas” de ese sistema, hay  $m$  variables  $x$  que son básicas y  $n - m$  no básicas,

Una idea, basada en la observación anterior, que permite aproximar el problema original de forma tratable consiste en interpretarlo como un problema de programación lineal, lo que se puede hacer aproximando  $\text{card}\{j : x_j \neq 0\} = \|x\|_0$  por  $\|x\|_1$ , que es la suma de los valores absolutos de los  $x$ :

$$\|x\|_1 = \sum_{j=1}^n |x_j|$$

De este modo, el problema de optimización queda como uno de programación lineal:

$$\begin{aligned} \text{mín} \quad & \sum_{j=1}^n |x_j| \\ \text{s.a.} \quad & Ax = y \end{aligned}$$

Este proxi ( $\text{card}\{j : x_j \neq 0\} = \|x\|_0 \approx \|x\|_1$ ) es mejor que utilizar  $\|x\|_2$ , pues tiende a hacer que en la medida de lo posible hallan entradas en  $x$  que se sean cercanas a cero, una intuición de porqué sucede esto se ve en la Figura 2.4.

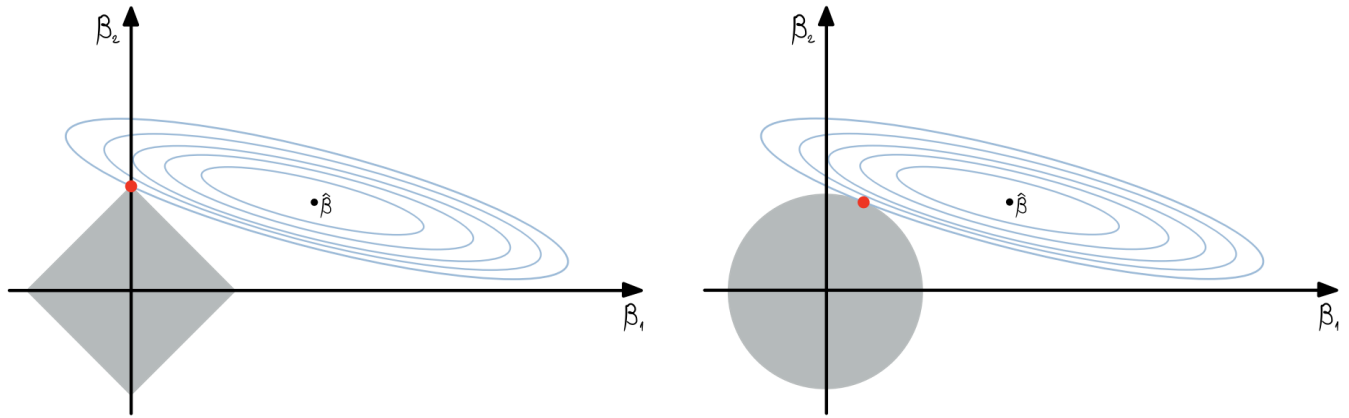


Figura 3.4: Norma uno más similar a la norma cero que la norma dos

Este segundo problema rescata el objetivo de que la mayor cantidad posible de entradas del vector  $x$  nulas. Como se recordará, en este tipo de problema, debido a que existen más variables (columnas de  $A$ ) que restricciones (filas de  $A$ ),  $m > n$ , entonces los óptimos de programación lineal, se alcanzan en los vértices, generan soluciones básicas que presentan gran cantidad de variables con valor  $= 0$ .

Sin embargo, aún este problema de optimización resulta difícil para  $n$  muy grande y la propuesta desarrollada es tratarlo de forma aproximada utilizando una “Regularización” en el problema de mínimos cuadrados que habíamos planteado anteriormente, que se traduce en el siguiente problema de optimización irrestricto:

$$\min_{x \in \mathbb{R}^n} \tau \|x\|_1 + \|Ax - b\|_2^2$$

donde  $\tau > 0$  es un parámetro dado. Este se conoce como un problema de mínimos cuadrados con “regularización L-1” y esconde dos funciones objetivo, que está balanceadas por medio del parámetro  $\tau$ . El primer término busca que la mayor cantidad posible de las entradas del vector  $x$  sean nulas. El segundo término minimiza el error de ajuste de  $x$ , ya que busca que  $Ax$  sea lo más parecido posible al vector  $b$ , es decir, el el tradicional término de mínimos cuadrados. Por lo tanto, si  $\tau$  es grande se tendrán pocas entradas de  $x$  distintas de cero pero un mal ajuste, mientras que si se tiene un  $\tau$  pequeño, se tendrá un buen ajuste, pero pocas variables distintas de cero. Es por esto que este valor se debe calibrar según el error que se quiera y la cantidad de genes o variables que expliquen el fenómeno.

### 3.2.2. El Método del Subgradiente

Ahora bien, uno de los problemas que tiene la formulación anterior es que la función objetivo es no diferenciable, pues  $\|x\|_1$  se define como:

$$\|x\|_1 = \sum_{j=1}^n |x_j|$$

función que no es diferenciable en el punto  $x = 0$ .

Como el problema es no diferenciable, es necesario recurrir a un concepto que juegue el mismo papel de las derivadas en los problemas diferenciables. Para esto, vamos a introducir algunas nociones básicas de análisis convexo no diferenciable, notablemente el concepto de subgradiente.

#### Definición 3.2.1: Concepto de Subgradiente

Sea  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  una función convexa. Un vector  $h \in \mathbb{R}^n$  es una subgradiente de  $f$  en  $x$  si:

$$f(y) \geq f(x) + h^T(y - x), \quad \forall y$$

La figura ilustra el significado del concepto. Un subgradiente es la “pendiente” de un hiperplano tangente que está por abajo del grafo de la función. En el caso de una función diferenciable, este coincide con el gradiente (la derivada), pero en un punto no diferenciable hay infinitos subgradientes que cumplen con la condición. En el caso de la figura, que es en una dimensión, es un rango de intervalo para valores. En el caso multidimensional, que es el de uso práctico, es un conjunto en el espacio multidimensional.

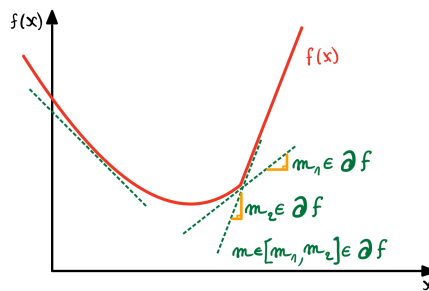
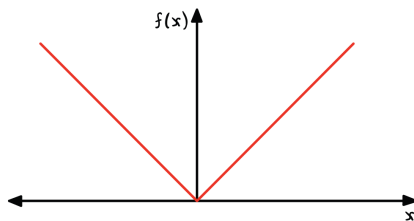


Figura 3.5: Subgradiente

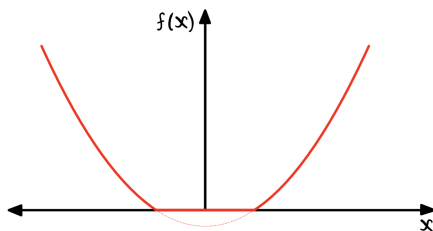
A continuación se muestran dos ejemplos de subgradientes:

El subgradiente de  $f(x) = |x|$  es el que se muestra a continuación:



$$h = \begin{cases} 1, & x > 0 \\ -1, & x < 0 \\ \alpha \in [-1, 1], & x = 0 \end{cases}$$

El subgradiente de  $f(x) = \max\{0, (x^2 - 1)\}$  se calcula considerando que la pendiente de dicha función es



$$h = \begin{cases} 2x, & x < -1 \vee x > 1 \\ 0, & -1 < x < 1 \\ \alpha \in [-2, 0], & x = -1 \\ \alpha \in [0, 2], & x = 1 \end{cases}$$

Con todo esto explicado, se define el Subdiferencial:

**Definición 3.2.2: Concepto de Subdiferencial**

Sea  $f : \mathbb{R}^n \rightarrow \mathbb{R}$  una función convexa. El conjunto  $\partial f(x)$  se le llama subdiferencial de  $f$  en  $x$ , tal que:

$$\partial f(x) = \{h \in \mathbb{R}^n : f(y) \geq f(x) + h^T(y - x), \forall y\}$$

El Subdiferencial  $\partial f(x)$  es, entonces, el conjunto de todos los subgradien- tes. Se puede argumentar fácilmente que si  $x^*$  es un mínimo global, entonces  $0 \in \partial f(x^*)$ .

**Cápsula de video 3.2.1: Subgradiente**

Profesor Jorge Vera explica en clases el concepto de subgradiente, con un ejemplo.

<https://youtu.be/-WBdyuIPJTI>

Con todo esto se puede definir el Método del Subgradiente, que utiliza como dirección de descenso no el negativo del gradiente (pues la función no diferenciable no tiene), sino que se utiliza una dirección que pertenezca al subdiferencial. Este Método del Subgradiente se presenta a continuación:

**Método del Subgradiente**

**Data:** Sea  $x^0 \in \mathbb{R}^n$ ,  $k = 0$ ,  $-d^k \in \partial f(x^0)$ ,  $\epsilon > 0$ .

**while** Criterio de parada no se satisface **do:**

Elegir el paso  $\lambda_k$   
 $x^{k+1} = x^k + \lambda_k d^k$   
 $-d^{k+1} \in \partial f(x^{k+1})$   
 $k = k + 1$

**end**

Una de las preguntas, por supuesto, es la determinación de un paso de avance adecuado. Haremos esa discusión al hablar de la convergencia del método más adelante.

**Ejemplo: Método del Subgradiente aplicado al problema LASSO**

Usemos el problema de LASSO para ver cómo funciona el Método del Subgradiente. En este caso, el problema de optimización es:

$$\min_{x \in \mathbb{R}^n} \tau \|x\|_1 + \|Ax - b\|_2^2$$

Para aplicar el algoritmo, se debe elegir una dirección de descenso  $-d^k \in \partial f(x^k)$ . Para esto se busca el subgradiente de  $f$ , haciendo abuso de notación, para esto se aplica la “subderivada” a la expresión. Como  $\|Ax - b\|_2^2$  es diferenciable, entonces su subderivada corresponde a su gradiente. De este modo, el subgradiente de  $f$  es el siguiente:

$$\begin{aligned} \partial f(x) &= \partial (\tau \|x\|_1) + \partial (\|Ax - b\|_2^2) \\ &= \tau \partial (\|x\|_1) + \nabla (\|Ax - b\|_2^2) \\ &= \tau \partial \left( \sum_{j=1}^n |x_j| \right) + 2A^T(Ax - b) \\ &= \tau \sum_{j=1}^n \partial |x_j| + 2A^T(Ax - b) \end{aligned}$$

Donde el valor de  $\partial |x_j|$  corresponde a un vector, llamémoslo  $p$ , cuyas entradas distintas de  $j$ ,  $p_{i \neq j}$ , son nulas (pues “subderiva” respecto al componente  $j$ ) y la que está en la posición  $j$  tiene la siguiente forma:

$$p_j = \begin{cases} 1, & x_j > 0 \\ -1, & x_j < 0 \\ \alpha \in [-1, 1], & x_j = 0 \end{cases}$$

Para este caso, elegir  $\alpha = 0$  es suficiente para encontrar un subgradiente. Una función que pertenece a  $p_j$  es la función signo,  $\text{sgn}(x)$ , cuya componente  $j$  se define del siguiente modo:

$$\text{sgn}(x)_j = \begin{cases} 1, & x_j > 0 \\ -1, & x_j < 0 \\ 0, & x_j = 0 \end{cases}$$

Luego, se puede utilizar como subgradiente válido la siguiente función:

$$\tau \text{sgn}(x) + 2A^T(Ax - b) \in \partial f(x)$$

Cabe decir que cuando se ejecuta el código de este problema, sucede que algunas entradas de  $x$  no convergen a cero perfectamente, sino que se acercan a números muy pequeños en valor absoluto, varios ordenes de magnitud más pequeños que las entradas de  $x$  que son definitivamente distintas de cero. Lo que se hace, en la práctica, es “redondear” estas entradas a 0 y así se puede medir  $\|x\|_0$ . El tema es ir calibrando el  $\tau$  de modo que se obtenga  $\|x\|_0$  cercano a la cantidad de columnas ( $\|x\|_0 \approx m$ ) y que el error de ajuste del sistema  $Ax = b$  no sea tan grande (15% es mucho, pero a veces es aceptable).

### Relación ente lipschitzianidad y subgradiente

Sea  $f$  función  $L$ -Lipschitz, entonces, si  $g \in \partial f(x) \rightarrow \|g\| \leq L$ . Esto se debe a que como  $f$  es función  $L$ -Lipschitz, entonces:  $\|f(y) - f(x)\| \leq L\|y - x\|$ . Además, sabemos que los subgradientes forman planos soporte de la función, es decir:

$$g \in \partial f(x) \rightarrow f(x) + g^T(y - x) \leq f(y)$$

Haciendo el cambio de variable:  $y = g + x$  se tiene:

$$\begin{aligned} f(x) + g^T g &\leq f(x + g) \\ \|g\|^2 &\leq f(x + g) - f(x) \\ \|g\|^2 &\leq \|f(x + g) - f(x)\| \quad / \text{Aplicando Lipschitz} \\ \|g\|^2 &\leq L\|g\| \\ \|g\| &\leq L \end{aligned}$$

Esta relación es útil para estimar la constante de Lipschitz de la función  $f$  teniendo una expresión para los subgradientes. Por ejemplo, en el caso de la función de regularización L1 de LASSO, teníamos que un subgradiente, en  $x$ , era:

$$g(x) = \tau \text{sgn}(x) + 2A^T(Ax - b)$$

Entonces, podemos estimar la constante de Lipschitz de la función estimando  $\|g(x)\|_2$ :

$$\|g(x)\|_2 \leq \tau \|\text{sgn}(x)\|_2 + 2\|A^T(Ax - b)\|_2 \leq \tau \sqrt{n} + 2\|A^T(Ax - b)\|_2$$

y para el segundo término, podemos estimar:

$$\|A^T(Ax - b)\|_2 \leq \|A^T A\|_2 \|x\|_2 + \|A^T b\|_2$$

Como se ve, esta expresión depende de  $x$ , haciendo pensar que no es posible determinar una constante de Lipschitz. Esto es verdad, efectivamente, pero sólo en un sentido global. Si pudiéramos restringirnos a un conjunto de valores de  $x$ , por ejemplo, conocer  $R$  tal que  $\|x\|_2 \leq M$ , entonces podemos estimar

$$L = \tau \sqrt{n} + 2\|A^T A\|_2 R + 2\|A^T b\|_2$$

En muchas aplicaciones es, efectivamente, posible asegurar que el espacio de búsqueda en las variables  $x$  tendrá esa propiedad y, entonces, sí se tendrá una constante de Lipschitz para la función objetivo del problema LASSO. Esto se ve reflejado, de hecho, en el resultado de convergencia que se presenta a continuación.

### Convergencia del Método del Subgradiente

Si es que suponemos que se conoce un  $R > 0$ , tal que  $f$  es convexa y  $L$ -Lipschitz en una vecindad  $R$  de  $x^*$ , es decir:

$$|f(x) - f(y)| \leq L\|x - y\|, \quad \forall x, y \in B(x^*, R)$$

(o sea se cumple Lipschitz en un espacio local cercano al objetivo). Si suponemos además que comenzamos en el punto  $x^0$  tal que  $x^0 \in B(x^*, R)$ , entonces se puede demostrar que si se elige un paso

$$\lambda_k = \frac{a}{\sqrt{k+1}}$$

Con  $a > 0$  una constante elegida juiciosamente, entonces:

$$\min_{0 \leq i \leq k} f(x^i) \leq f(x^*) + \frac{R^2 + a \ln(k+1)}{2a\sqrt{k+1}}$$

Es  $\min_{0 \leq i \leq k} f(x^i)$  en vez de  $f(x^k)$  porque este método no garantiza que en cada iteración se descienda. Esta expresión es mala, pues implica que para tener un error pequeño se debe iterar muchísimo ( $k$  muy grande). Esto se puede hacer mejor si se elige que en el paso, la constante  $a$  sea  $R/L$ :

$$\lambda_k = \frac{R}{L\sqrt{k+1}}$$

Con esto, se tiene que:

$$\min_{0 \leq i \leq k} f(x^i) \leq f(x^*) + \frac{RL}{\sqrt{k+1}}$$

El gran problema es que se necesita conocer  $R$  y el  $L$  para calcular esto, pero como ya dijimos, en algunas aplicaciones sí es posible obtenerlos. El segundo término del lado derecho de la desigualdad es el error, por lo que si se ejecuta el algoritmo hasta alcanzar un error inferior a  $\epsilon$ , entonces el número de iteraciones necesarios tiene por cota inferior:

$$\frac{RL}{\sqrt{k+1}} \leq \epsilon$$

$$k \geq \left(\frac{RL}{\epsilon}\right)^2 - 1$$

Por lo que la complejidad del algoritmo es del orden de  $\mathcal{O}(1/\epsilon^2)$ , mucho peor que  $\mathcal{O}(1/\epsilon)$  del caso diferenciables (Ya que  $\epsilon < 1$ ). El caso diferenciable es mejor porque en el fondo tiene derivadas que le dan estructura, recordar que las propiedades lipschitzianas se las debíamos al gradiente, que era una manera de aproximarse al segundo orden, cosa que no se puede hacer con el subgradiente, que utiliza (valga la redundancia) el subgradiente, que no tiene tanta estructura e información como lo es el gradiente. Ante esta mala noticia, cabe decir que de todos modos, al igual que en el caso diferenciable, se puede acelerar el algoritmo, tal como lo hace el método FISTA que se presenta más adelante.

El siguiente código muestra el algoritmo de subgradiente aplicado al problema de LASSO.

```
import numpy as np

def LASSO(A, b, tau, iteracion_maxima):
    """
    Funcion que busca resolver el problema de regulacion L1 mediante el metodo del
    subgradiente. Su entrada posee:
        - La matriz A, el vector b y el escalar tau de trade off de la regularizacion L1.
```

```

- iteracion_maxima: Numero maximo de iteraciones a realizar.
Su salida es:
- valor: Valor optimo del problema a minimizar.
- xk: Vector solucion del problema de k iteraciones.
"""
# Dimensiones correspondientes
m, n = A.shape
# Vector con ceros en todas sus entradas como vector inicial de partida
xk = np.zeros((n, 1))
# Estimacion del valor de R (radio) y L (constante de Lipschitz)
R = np.sqrt(np.linalg.norm(np.linalg.inv(
    np.dot(A, np.transpose(A))))*np.linalg.norm(b))
L = tau*np.sqrt(n) + np.linalg.norm(np.dot(A, np.transpose(A))) * \
    R + np.linalg.norm(np.dot(np.transpose(A), b))
# Numero de iteraciones segun el teorema de la convergencia
thetak = R/(L*np.sqrt(iteracion_maxima+1))

# OJO OJO: Se puede usar thetak = a/ np.sqrt(iteracion+1)

k = 0
while k <= iteracion_maxima:
    # 1: Se calcula el gradiente de la funcion objetivo
    subgradiente = 2*np.dot(np.transpose(A),
        np.dot(A, xk) - b) + tau*np.sign(xk)

    # 2: Se actualiza el valor objetivo
    xk = xk - thetak*subgradiente

    # 3: Se evalua el error de ajuste
    error = np.amax(np.abs(np.dot(A, xk) - b)) / np.amax(np.abs(b))
    if error <= 0.0001: break

    k += 1

valor = np.linalg.norm(np.dot(A, xk) - b, 2)**2 + tau*np.linalg.norm(xk, 1)

return valor, xk

if __name__ == '__main__':
    np.random.seed(2121)
    A, b = generar_datos_LASSO(30, 100)

    valor, x = LASSO(A, b, 10, 10000)

```

### Cápsula de video 3.2.2: El Método del Subgradiente aplicado al problema LASSO

Profesor Jorge Vera muestra el desarrollo del Método del Subgradiente para el problema de estimación LASSO, con resultados computacionales usando Octave, que es un “sustituto” libre de MATLAB.  
<https://youtu.be/T8AvSRaBlow>

### 3.2.3. FISTA: Fast Iterative Shrinkage and Thresholding Algorithm

Este es un método que fue desarrollado por A.Beck y M.Teboulle e implementa un concepto de aceleración similar al de Nesterov, utilizando dos sucesiones de puntos. Es muy usado para la reconstrucción de imágenes y problemas de Compressed Sensing. Tomemos el problema no diferenciable LASSO, ya estudiado:

$$\min_{x \in \mathbb{R}^n} \tau \|x\|_1 + \|Ax - b\|_2^2$$

donde el segundo término de la función objetivo, que es diferenciable, se puede reescribir como  $h(x) = \|Ax - b\|_2^2$ , de modo que el problema queda como:

$$\min_{x \in \mathbb{R}^n} \tau \|x\|_1 + h(x)$$

Si  $L$  es la constante de Lipschitz de  $\nabla h(x)$ , entonces el algoritmo se puede describir del siguiente modo:

---

**FISTA: Fast Iterative Shrinkage-Thresholding Algorithm**


---

**Data:**  $x^0, z^0 \in \mathbb{R}^n, k = 0, \theta_0 = 1$ .

**while** *Criterio de parada no se satisface* **do:**

$$y_k = z_k - \frac{1}{L} \nabla h(z_k)$$

Resolver:

$$\min_x \left\{ \tau \|x\|_1 + \frac{L}{2} \|x - y_k\|_2^2 \right\}$$

sea  $x_{k+1}$  la solución óptima.

$$\text{Sea } \theta_{k+1} = \frac{1}{2} \left( 1 + \sqrt{1 + 4\theta_k^2} \right).$$

$$z_{k+1} = x_{k+1} + \left( \frac{\theta_k - 1}{\theta_{k+1}} \right) (x_{k+1} - x_k)$$

$$k = k + 1$$

**end**

---

La complejidad de este algoritmo es la misma que para el método acelerado de Nesterov, pues para llegar a una solución  $\epsilon$ -óptima se requieren del orden de  $\mathcal{O}(1/\sqrt{\epsilon})$  iteraciones.

El problema de minimización que se debe resolver en cada iteración de este problema es, en realidad, muy simple de resolver porque se puede descomponer coordenada a coordenada del siguiente modo:

$$\min_x \left\{ \tau \sum_{j=1}^n |x_j| + \frac{L}{2} \sum_{j=1}^n (x_j - (y_k)_j)^2 \right\}$$

Luego, para la coordenada  $j \in \{1, \dots, n\}$  el problema es:

$$\min_{x_j} \left\{ \tau |x_j| + \frac{L}{2} (x_j - (y_k)_j)^2 \right\}$$

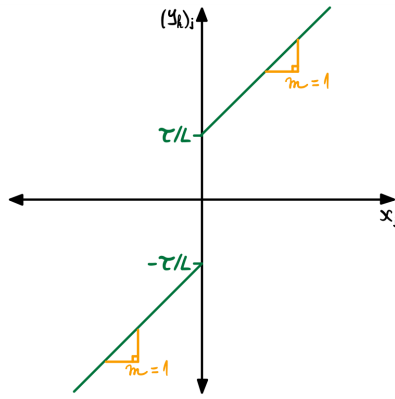
Dado que es una función claramente convexa, basta derivar e igualar a cero para encontrar el punto  $x_j^*$  donde se alcanza el mínimo:

$$\tau \cdot \text{sgn}(x_j^*) + L (x_j^* - (y_k)_j) = 0$$

Resolviendo se obtiene:

$$(y_k)_j = x_j^* + \frac{\tau}{L} \text{sgn}(x_j^*)$$

La forma del espacio geométrico conformado por las variables  $(y_k)_j$  y  $x_j$  conforma una función de tipo *Soft Thresholding*, la que se muestra en la siguiente figura



$$x_j = \begin{cases} y - \tau/L, & (y_k)_j \geq \tau/L \\ y + \tau/L, & (y_k)_j \leq -\tau/L \\ 0, & \text{e.o.c.} \end{cases}$$

Equivalentemente, se puede expresar lo mismo como:

$$x_j = \text{sgn}((y_k)_j) \cdot \max \left\{ |(y_k)_j| - \frac{\tau}{L}, 0 \right\}$$

El siguiente es un código Python que implementa el algoritmo FISTA:

```
import numpy as np
from apendice import generar_datos

def LASSO_FISTA(A, b, tau, iteracion_maxima):
    """
    Esta funcion implementa el algoritmo de FISTA para resolver el problema de
    regularizacion L1: min_x {tau * ||x||_1 + ||Ax - b||_2^2}. Su entrada posee:
    - A, b, tau: Parametros propios del problema.
    - iteracion_maxima : Numero maximo de iteraciones a realizar.
    Su salida es:
    - valor : Valor optimo del problema a minimizar.
    - xk : Vector solucion del problema de k iteraciones.
    """
    def xk_optimo_FISTA(tau, L, y):
        """
        Funcion que optimiza el problema de FISTA:
        min_x {tau * ||x||_1 + (L / 2) * ||x - yk||_2^2}
        """
        x_r = np.zeros((len(y), 1))
        for j in range(len(y)):
            if y[j] >= (tau / L): x_r[j] = y[j] - (tau / L)
            elif y[j] <= -(tau / L): x_r[j] = y[j] + (tau / L)
            else: x_r[j] = 0 * y[j]
        return x_r

    # Se setean las dimensiones
    m, n = A.shape
    # Se setean las condiciones iniciales, junto a los vectores que almacenando
    xk, zk, thetak = np.zeros((n, 1)), np.zeros((n, 1)), 1
    # Estimacion del valor de R (radio) y L (connstante de Lipschitz)
    R = np.sqrt(np.linalg.norm(np.linalg.inv(np.dot(A, np.transpose(A))))
                * np.linalg.norm(np.dot(np.transpose(A), b)))
    L = tau * np.sqrt(n) + np.linalg.norm(np.dot(A, np.transpose(A))) * \
        R + np.linalg.norm(np.dot(np.transpose(A), b))
    k = 0

    while k <= iteracion_maxima:
        # 1: se calcula y_{k}
        yk = zk - (1 / L) * 2 * np.dot(np.transpose(A), np.dot(A, zk) - b)

        # 2: se calcula x_{k+1}
        # Alternativamente se puede resolver con Soft Thresholding
        xk1 = xk_optimo_FISTA(tau, L, yk)
```

```

# 3: se calcula theta_{k+1}
thetak1 = (1 / 2) * (1 + np.sqrt(1 + 4* thetak ** 2))

# 4: Se calcula el paso z_{k+1}
zk1 = xk1 + ((thetak - 1) / thetak1) * (xk1 - xk)

# 5: Se actualizan los valores para la siguiente iteracion
k, zk, xk, thetak = k + 1, zk1, xk1, thetak1

valor = tau * np.linalg.norm(xk, 1) + 0.5*np.linalg.norm(np.dot(A, xk) - b)**2

return valor, xk

if __name__ == '__main__':
    np.random.seed(2121)
    tau, iteracion_maxima = 10,800
    A, b = generar_datos(100, 500)

    valor, xsol = LASSO_FISTA(A, b, tau, iteracion_maxima)

```

### Cápsula de video 3.2.3: Algoritmo FISTA

Profesor Jorge Vera explica en clases el algoritmo FISTA y se compara computacionalmente con el Método de Subgradiente simple.  
[https://youtu.be/L-UT\\_OTq\\_2k](https://youtu.be/L-UT_OTq_2k)

### 3.2.4. Resumen

Hemos estudiado hasta ahora algunos métodos para problemas irrestrictos y que pueden abordar funciones objetivo diferenciables y no diferenciables. Según las hipótesis que se impongan, pueden tener distintas complejidades. La siguiente tabla resume las complejidades en el número de iteraciones para llegar a una solución  $\epsilon$  óptima con los Métodos de Primer Orden estudiados hasta ahora:

| Problema                                   | Método Simple                                     | Método Acelerado                                    |
|--------------------------------------------|---------------------------------------------------|-----------------------------------------------------|
| $f$ diferenciable, $\nabla f$ L-Lipschitz  | $\mathcal{O}\left(\frac{1}{\epsilon}\right)$      | $\mathcal{O}\left(\frac{1}{\sqrt{\epsilon}}\right)$ |
| $f$ no diferenciable, $f$ L-Lipschitz      | $\mathcal{O}\left(\frac{1}{\epsilon^2}\right)$    | $\mathcal{O}\left(\frac{1}{\sqrt{\epsilon}}\right)$ |
| $f$ fuerte convexa, $\nabla f$ L-Lipschitz | $\mathcal{O}\left(\log \frac{1}{\epsilon}\right)$ | $\mathcal{O}\left(\log \frac{1}{\epsilon}\right)$   |

Hay un teorema de Nesterov y otro de Nemirovsky que dicen que si no hay convexidad fuerte, entonces con el método acelerado se pueden obtener los mejores rendimientos y esos están limitados al orden  $= (1/\sqrt{\epsilon})$ . Esto es, entonces, una “cota inferior” a lo que podemos lograr con conocimiento de un oráculo de primer orden.

## 3.3. Métodos de Primer Orden con Restricciones

Los métodos que hemos estudiado hasta ahora asumen problemas de la forma:

$$\min_{x \in \mathbb{R}^n} f(x)$$

Donde  $f$  es convexa, y puede ser o no diferenciable. La idea de un Método de Primer Orden se puede extender a problemas con restricciones, como veremos ahora. Conceptualmente, la idea es relativamente simple pero la eficiencia de la idea depende fuertemente de las características del problema.

Consideremos el siguiente problema:

$$\begin{aligned} \min \quad & f(x) \\ \text{s.a.} \quad & x \in D \end{aligned}$$

Donde  $D$  es convexo. Vamos a revisar en las siguientes secciones, primero, una idea general que se conoce como Método Proyectado y otra, que corresponde a un algoritmo antiguo pero que ha tenido un fuerte resurgimiento en años recientes, el Algoritmo de Frank-Wolfe.

### El Método Proyectado

La proyección de un punto sobre un conjunto consiste en buscar aquel punto de un conjunto que está a menor distancia de este. Denotemos por  $\Pi_D(x)$  la proyección de  $x$  en  $D$ . Esto corresponde a resolver el siguiente problema:

$$\begin{aligned} \Pi_D(x) = \min \quad & \|x - y\|_2 \\ \text{s.a.} \quad & y \in D \end{aligned}$$

Así, el problema de optimización busca minimizar la distancia entre el punto  $x$  y el punto  $y \in D$ .

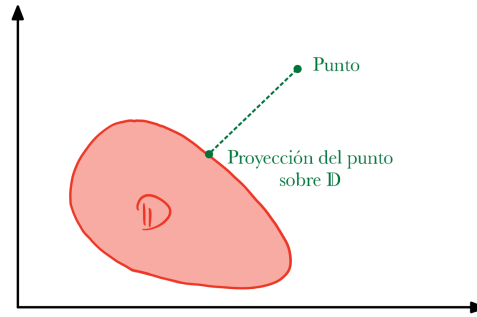


Figura 3.6: Método Proyectado

El Método Proyectado se desarrolla, simplemente, adaptando la etapa de avance a:

$$x^{k+1} = \Pi_D(x^k + \lambda_k d^k)$$

Con  $d^k$  la dirección de descenso, y  $\lambda_k$  el paso. De este modo, en primer lugar se hace un paso  $x^k + \lambda_k d^k$  y luego se proyecta sobre el espacio factible  $D$  para así obtener el nuevo punto  $x^{k+1} \in D$ . De este modo, el algoritmo es el siguiente:

#### Algoritmo simple de primer orden proyectado

**Data:**  $x^0 \in \mathbb{R}^n, k = 0, d^0 = -\nabla f(x^0)$ .  
**while** *Criterio de parada no se satisface* **do:**  
     $x^{k+1} = \Pi_D(x^k + \lambda^k d^k)$   
     $d^{k+1} = -\nabla f(x^{k+1})$   
     $k = k + 1$   
**end**

Este método también es aplicable a funciones no diferenciables, para lo que se debe tomar la dirección de descenso  $-d^k \in \partial f(x^k)$ . También se puede aplicar al método acelerado e incluso a FISTA. Cabe decir que las convergencias son iguales al caso sin restricciones y se usan los mismos  $\lambda$  para los pasos.

Todo esto, desde un punto de vista práctico, solo tiene sentido si el conjunto  $D$  es “simple”, en el sentido que se pueda calcular la proyección sin un esfuerzo computacional adicional significativo.

Un ejemplo en que puede ser simple la proyección es en el problema LASSO abordado de manera alternativa:

$$\begin{aligned} \min \quad & \|Ax - b\|_2^2 \\ \text{s.a.} \quad & \|x\|_1 \leq \rho \end{aligned}$$

En este problema, el conjunto  $D$  tiene la siguiente forma:  $D = \{x : \|x\|_1 \leq \rho\}$ . En un poliedro así, es muy fácil proyectar. Véase la Figura 2.7 que representa a dicho poliedro en dos dimensiones:

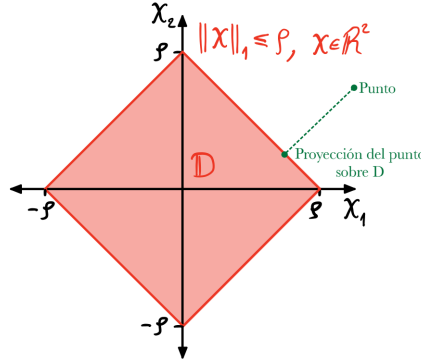


Figura 3.7: Poliedro sobre el que resulta sencillo proyectar

En el caso de encontrar la proyección del punto rojo sobre el poliedro  $D = \{x : \|x\|_1 \leq \rho\}$ , basta buscar solo sobre los vértices que se encuentran delimitando el cuadrante donde está el punto, vale decir, los puntos  $(0, \rho)$  y  $(\rho, 0)$ , luego se debe definir la línea que une a estos dos puntos:  $x_1 + x_2 = \rho$  (en  $\mathbb{R}^n$  correspondería a un hiperplano), luego se proyecta el punto sobre la línea, de modo que resulte sencillo ver cual es el vértice más cercano.

### Algoritmo de Frank - Wolfe

Algoritmo para resolver problemas de optimización con restricciones muy usado en lo que concierne a Equilibrios en Redes de Transporte y en Economía de Transporte. Es utilizado para abordar problema del siguiente tipo:

$$\begin{array}{ll} \text{mín} & f(x) \\ \text{s.a.} & Ax \leq b \end{array}$$

Si  $f$  fuera lineal, el problema es simple de resolver, utilizando el algoritmo SIMPLEX de Programación Lineal. Si el  $f$  fuera no lineal, entonces se puede aproximar a una función lineal en  $y$  y se optimiza esta. Se linealiza la función objetivo, suponiendo que estamos en un punto  $x^k$ :

$$f(y) \approx f(x^k) + \nabla f(x^k)^T (y - x^k)$$

La propuesta es minimizar dicha aproximación lineal, por lo que el problema a resolver queda de la siguiente forma:

$$\begin{array}{ll} \text{mín} & \nabla f(x^k)^T (y - x^k) \\ \text{s.a.} & Ay \leq b \end{array}$$

Como  $\nabla f(x^k)^T x^k$  es una constante, el problema de optimización se puede simplificar del siguiente modo:

$$\begin{array}{ll} \text{mín} & \nabla f(x^k)^T y \\ \text{s.a.} & Ay \leq b \end{array}$$

Sea  $\bar{y}$  la solución óptima de este problema. Luego el punto de la siguiente iteración se elije del siguiente modo:

$$x^{k+1} = x^k + \alpha_k (\bar{y} - x^k)$$

Aquí, las alternativas para un  $\alpha$  adecuado son:

1. Hacer un *linesearch*:  $\min_{0 \leq \alpha \leq 1} f(x^k + \alpha(\bar{y} - x^k))$
2. Usar un valor definido a priori, cosa más utilizado hoy en día debido a que es más “barato” en cada iteración. Para esto necesitamos suponer que  $f$  es convexa, y que  $\nabla f$  cumple una condición de Lipschitz, tal que:  $\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|$ ,  $\forall \{x, y\} \in D$ . El siguiente paso queda definido como:

$$\alpha_k = \frac{2}{k+2}$$

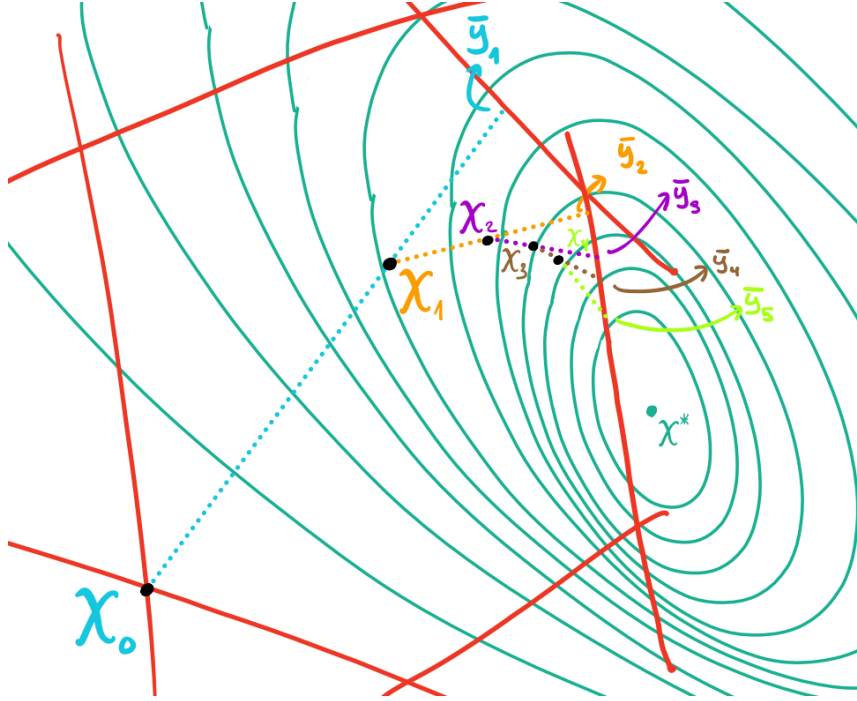


Figura 3.8: Ejemplo de 4 iteraciones con Frank-Wolfe

El algoritmo en cuestión se presenta a continuación:

#### Algoritmo de Frank-Wolfe (paso simple)

**Data:**  $x^0 : Ax^0 \leq b, k = 0, d^0 = -\nabla f(x^0)$ .

**while** Criterio de parada no se satisface **do**:

Sea  $\bar{y}$  solución óptima del siguiente problema:

$$\begin{aligned} \text{mín} \quad & \nabla f(x^k)^T (y - x^k) \\ \text{s.a.} \quad & Ay \leq b \end{aligned}$$

$$\begin{aligned} x^{k+1} &= x^k + \frac{2}{k+2} (\bar{y} - x^k) \\ k &= k + 1 \end{aligned}$$

**end**

En general, como criterio de parada se utilizan las condiciones de Karush Kuhn Tucker. Este algoritmo puede llegar a ser muy eficiente en problemas en donde el subproblema lineal puede ser especialmente fácil. Esto sucede si reformulamos aproximadamente el problema de LASSO, con su Regularización-L1:  $\min\{\tau\|x\|_1 + \|Ax - b\|_2^2\}$  en el siguiente problema que minimiza el ajuste sujeto a que la norma uno de  $x$  sea menor a un parámetro adecuado  $\rho$ :

$$\begin{aligned} \text{mín} \quad & \|Ax - b\|_2^2 \\ \text{s.a.} \quad & \|x\|_1 \leq \rho \end{aligned}$$

Notar que la función objetivo es convexa, su gradiente cumple con la propiedad de Lipschitz y la restricción es lineal. Para el problema de Regularización-L1, Frank-Wolfe presenta una ventaja muy interesante, ya que como en cada iteración resuelve el problema:

$$\begin{aligned} \text{mín} \quad & \nabla f(x^k)^T (y - x^k) \\ \text{s.a.} \quad & \|y\|_1 \leq \rho \end{aligned}$$

entonces, como la solución óptima de este sub-problema se encuentra en un vértice del poliedro  $D$ , definido como  $D = \{x : \|y\|_1 \leq \rho\}$ , resulta que este problema es particularmente sencillo de resolver con Frank-Wolfe, pues la restricción del poliedro  $D$  se puede reescribir como:

$$\begin{aligned} -t_j &\leq y_j \leq t_j, \quad \forall j \\ \sum_{j=1}^n t_j &\leq \rho \\ t_j &\geq 0 \end{aligned}$$

Esta restricción se ilustra en la Figura 2.7. En este caso, los vértices son tales que el óptimo se encuentran en puntos en que todas sus coordenadas son nulas salvo una que es  $\rho$  o  $-\rho$ , por lo que no es necesario ejecutar SIMPLEX, sino simplemente recorrer el vector y ver en que entrada conviene que esté el  $\rho$  o  $-\rho$ . De este modo, el algoritmo de Frank-Wolfe va a comenzar con el vector  $x^0 = 0$  (vector nulo) que es evidentemente factible al estar en el centro del espacio  $D$  y en el vector  $x^1$  resultante de la primera iteración solo una entrada va a ser distinta de cero cuando se le suma el  $\bar{y}$  resultante de resolver el problema como se plantea acá ( $x^1 = x^0 + \alpha_1(\bar{y} - x^0)$ ). Del mismo modo, en la segunda iteración nuevamente se va a agregar una nueva componente distinta de cero al vector y así sucesivamente, de modo que va a ser posible iterar hasta alcanzar una solución  $x$  en que una determinada cantidad de entradas sean distintas de cero. Esto es algo muy apropiado, pues lo que se buscaba originalmente era tener una solución con pocas entradas distintas de cero, y nuestro método va agregando entradas distintas de cero poco a poco.

La mala noticia es que en realidad, este algoritmo no es tan rápido. Hay un Teorema de Convergencia que dice que si se usa el paso definido a priori  $\alpha_k = \frac{2}{k+2}$ , y  $\{x^k\}_k$  es la sucesión de puntos generados por el algoritmo, entonces:

$$f(x^k) \leq f(x^*) + \frac{2L(\text{diam}(D))^2}{k+2}$$

Donde  $\text{diam}(D)$  es el “diámetro” del conjunto  $D$  (Cuánto mide el ancho más grande de  $D$ ). Esto significa que para llegar a un error  $\epsilon > 0$ , necesitamos un orden de  $\mathcal{O}(\frac{1}{\epsilon})$  iteraciones. Lo bueno es que esto se puede acelerar con *away steps*. Además, si existe convexidad fuerte esto llega a tener convergencia lineal. Nótese que el  $L$  no se utiliza en el algoritmo, entra solo en el teorema para justificar el error, pero no en la ejecución de este.

#### Cápsula de video 3.3.1: Algoritmo de Frank-Wolfe

Profesor Jorge Vera explica en clases los detalles del Algoritmo de Frank-Wolfe.  
[https://youtu.be/EqI118uOu\\_Q](https://youtu.be/EqI118uOu_Q)

## 3.4. Ejemplos

### 3.4.1. Función no diferenciable restringida

Consideremos el siguiente problema de optimización en  $n$  variables:

$$\begin{aligned} \text{mín} \quad & \sum_{j=1}^n \max\{0, (x_j - \alpha_j)\} + \sum_{j=1}^n x_j \log x_j \\ \text{s.a.} \quad & 1 \leq x_j \leq 2, \quad j = 1, \dots, n \end{aligned}$$

donde los  $\alpha_j$  son parámetros escalares tales que  $1 < \alpha_j < 2$ , para  $j = 1, \dots, n$ . Variantes de funciones como estas aparecen en algunos problemas de estimación estadística. La segunda parte de la función objetivo es una función de entropía. Denotemos por  $f$  la función objetivo del problema. Esta función es convexa.

Queremos abordar este problema de optimización con un Método de Primer Orden. Como la función objetivo es no diferenciable y, además, hay restricciones, desarrollaremos el Método de Subgradiente Proyectado para este caso particular.

Primero, vamos a determinar una expresión general para  $\partial f(x)$  en cualquier punto  $x$  relevante (donde esté definido el problema).

Por comodidad de notación, llamemos  $g(x) = \sum_{j=1}^n \max\{0, (x_j - \alpha_j)\}$  y  $h(x) = \sum_{j=1}^n x_j \log x_j$  de modo que la función objetivo sea  $f = g + h$ . Tenemos que  $g$  es no diferenciable y  $h$  es diferenciable. Se puede ver de inmediato que

$$\frac{\partial}{\partial x_j} h(x) = \log x_j + 1$$

y eso determina  $\nabla h(x)$ . Por otro lado, para el subdiferencial de  $g$  se ve que  $q \in \partial g(x)$  si tiene la siguiente forma:

$$q_j = \begin{cases} 0 & x_j < \alpha_j \\ 1 & x_j > \alpha_j \\ \gamma \in [0, 1] & x_j = \alpha_j \end{cases}$$

Luego,  $\partial f(x) = \partial g(x) + \{\nabla h(x)\}$ .

Ahora desarrollamos los pasos principales del método. Vamos a usar el método de subgradiente con el paso “eficiente” mostrado en el teorema de convergencia. Para el algoritmo necesitamos, entonces, explícitamente un subgradiente y necesitamos los parámetros  $R$  y  $L$ .

Como subgradiente en  $x$  vamos a tomar  $p(x) = \max\{0, \text{sgn}(x)\} + \nabla h(x)$ , que está basado en el desarrollo de a).

Para  $R$ , simplemente notemos que todo ocurre dentro de la caja definida por  $1 \leq x_j \leq 2$ ,  $j = 1, \dots, n$  así que, por ejemplo,  $R = 2\sqrt{n}$  es un número válido. También es más preciso  $R = \sqrt{n}$  (basado en el diámetro de la caja en  $n$  dimensiones, ya que todos los puntos relevantes estarán ahí), cualquiera de estos son valores aceptables.

Para  $L$  vamos a usar el resultado que dice que podemos estimar la constante de Lipschitz buscando una cota superior a  $\|p\|$ , donde  $p$  es un subgradiente. Ahora, usemos el subgradiente propuesto y, recordando que todo pasa dentro de la caja:

$$\|p\|_2 \leq \|\max\{0, \text{sgn}(x)\}\|_2 + \|\nabla h(x)\|_2$$

Ahora, observamos que  $\|\max\{0, \text{sgn}(x)\}\|_2 \leq \sqrt{n}$  y que para las derivadas parciales tenemos que, dentro de la caja:

$$\left| \frac{\partial}{\partial x_j} h(x) \right| \leq \log 2 + 1$$

así que  $\|\nabla h(x)\|_2 \leq \sqrt{n}(1 + \log 2)$  y esto da el valor de  $L$ .

El algoritmo sería el siguiente:

**Data:** Sea  $x^0 \in \mathbb{R}^n$ ,  $k = 0$ ,  $-d^0 \in \partial f(x^0)$ ,  $\epsilon > 0$ .

**while** Criterio de parada no se satisface **do**

$x^{k+1} = \Pi_D(x^k + \lambda^k d^k);$   
 $-d^{k+1} \in \partial f(x^{k+1});$   
 $k = k + 1;$

**end**

haciendo  $N$  iteraciones y usando un paso  $\lambda^k = \frac{R}{L\sqrt{N+1}}$ . En el algoritmo,  $D$  es la caja.

Discutamos ahora cómo se aborda el problema de proyección sobre la caja. Recordemos que es fundamental, para que todos estos algoritmos tengan real sentido práctico, que esta etapa de proyección se pueda hacer en forma realmente eficiente.

Para esto es más conveniente, para propósitos ilustrativos, pensar en la caja  $0 \leq x_j \leq 1, j = 1, \dots, n$ . Si queremos proyectar el punto  $u \in \mathbb{R}^n$ , lo primero que tenemos que hacer es determinar si acaso está o no dentro de los rangos 0-1 en alguna coordenada. Si  $0 \leq u_j \leq 1$ , entonces esa coordenada queda igual, es decir. Por otro lado, si  $u_j < 0$ , esa coordenada se debe poner en 0 y si  $u_j > 1$ , se debe llevar a 1. En resumen:

$$\Pi_D(u)_j = \begin{cases} u_j & 0 \leq u_j \leq 1 \\ 0 & u_j < 0 \\ 1 & u_j > 1 \end{cases}$$

Todo se puede escribir igual para la caja del problema, pero corrido en 1.

### 3.4.2. Subgradiente de la función “Max”

Consideremos la siguiente función en  $n$  variables:

$$f(x) = \max_{i=1, \dots, m} \{a_i^T x + b_i\}$$

donde  $a_i \in \mathbb{R}^n, i = 1, \dots, m, b_i \in \mathbb{R}$ .

Vamos a ver que  $f$  es convexa en el dominio en que está definida. Para esto el argumento se puede hacer tomando una combinación  $\lambda x + (1 - \lambda)y$  y evaluar:

$$\begin{aligned} \max_{i=1, \dots, m} \{a_i^T (\lambda x + (1 - \lambda)y) + b_i\} &= \max_{i=1, \dots, m} \{\lambda a_i^T x + (1 - \lambda)a_i^T y + b_i\} \\ &= \max_{i=1, \dots, m} \{\lambda(a_i^T x + b_i) + (1 - \lambda)(a_i^T y + b_i)\} \\ &\leq \lambda \max_{i=1, \dots, m} \{a_i^T x + b_i\} + (1 - \lambda) \max_{i=1, \dots, m} \{a_i^T y + b_i\} \end{aligned}$$

donde en la última relación hemos usado el hecho de que el máximo de una suma se acota superiormente por la suma de los máximos. Esto prueba la convexidad del primer sumando.

Ahora vamos a desarrollar una expresión para  $\partial f(x)$  en cualquier punto del dominio de  $f$ . Sea  $J$  el conjunto de índices donde se alcanza  $\max_{i=1, \dots, m} \{a_i^T x + b_i\}$ , es decir,

$$\max_{i=1, \dots, m} \{a_i^T x + b_i\} = a_j^T x + b_j, j \in J.$$

Eventualmente, puede tenerse que  $J$  contenga sólo un índice, pero los puntos donde  $f$  es no diferenciable son aquellos donde hay más de un índice donde se alcanza el máximo. Entonces, consideremos cualquier vector  $h$  perteneciente a

$$\text{conv}\{a_j, j \in J\}$$

Es fácil mostrar que  $h$  cumple la definición de subgradiente, es decir:

$$f(y) \geq f(x) + h^T(y - x)$$

En efecto, si  $h = \sum_{j \in J} \lambda_j a_j$ , con  $\sum_{j \in J} \lambda_j = 1$ , tenemos:

$$f(x) + \sum_{j \in J} \lambda_j a_j^T (y - x) = f(x) + \sum_{j \in J} \lambda_j a_j^T y - \sum_{j \in J} \lambda_j a_j^T x$$

Pero recordemos que el máximo se alcanza en los índices  $J$  y todo los términos  $a_j^T x + b_j$  son iguales al

máximo. Entonces:

$$\begin{aligned}
 f(x) + \sum_{j \in J} \lambda_j a_j^T (y - x) &= f(x) + \sum_{j \in J} \lambda_j a_j^T y - \sum_{j \in J} \lambda_j (a_j^T x + b_j - b_j) \\
 &= f(x) + \sum_{j \in J} \lambda_j (a_j^T y + b_j) - \sum_{j \in J} \lambda_j (a_j^T x + b_j) \\
 &= f(x) + \sum_{j \in J} \lambda_j (a_j^T y + b_j) - \sum_{j \in J} \lambda_j f(x) \\
 &= f(x) + \sum_{j \in J} \lambda_j (a_j^T y + b_j) - f(x) \\
 &= \sum_{j \in J} \lambda_j (a_j^T y + b_j) \\
 &\leq \sum_{j \in J} \lambda_j \max\{(a_j^T y + b_j), j = 1, \dots, m\} \\
 &= f(y)
 \end{aligned}$$

lo que prueba que  $h$  es, efectivamente, un subgradiente.

### 3.4.3. Subgradiente de función “max” combinadas con otras

Considere la siguiente función definida en  $\mathbb{R}^n$ :

$$f(x) = \sum_{j=1}^n \{x_j^2 + \max\{0, \alpha_j x_j + b_j\}\}$$

donde los  $\alpha_j$  y  $b_j$  son escalares, con  $\alpha_j \neq 0$ ,  $j = 1, \dots, n$ . Vamos a determinar el subdiferencial de  $f$  en cualquier punto  $x$  del dominio.

Se quiere calcular  $\partial(f(x))$

$$\begin{aligned}
 &= \partial\left(\sum_{j=1}^n \{x_j^2 + \max\{0, \alpha_j x_j + b_j\}\}\right) \\
 &= \partial\left(\sum_{j=1}^n x_j^2\right) + \partial\left(\sum_{j=1}^n \max\{0, \alpha_j x_j + b_j\}\right) \\
 &= \partial(x_1^2 + x_2^2 + x_3^2 + \dots + x_n^2) + \partial\left(\sum_{j=1}^n \rho_j(x)\right) \\
 &= \nabla(x^T x) + \rho(x) \\
 &= 2x + \rho(x)
 \end{aligned}$$

con  $x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$  y  $\rho(x)$  un vector  $\in \mathbb{R}^n$ , cuyo  $j$ -ésimo elemento es  $\rho_j(x)$  y se define como:

$$\begin{aligned}
 \text{Si } \alpha_j < 0 : \quad \rho_j(x) &= \begin{cases} 0 & \text{si } 0 > \alpha_j x_j + b_j \\ \alpha_j & \text{si } 0 < \alpha_j x_j + b_j \\ t \in [\alpha_j, 0] & \text{si } 0 = \alpha_j x_j + b_j \end{cases} \\
 \text{Si } \alpha_j > 0 : \quad \rho_j(x) &= \begin{cases} 0 & \text{si } 0 > \alpha_j x_j + b_j \\ \alpha_j & \text{si } 0 < \alpha_j x_j + b_j \\ t \in [0, \alpha_j] & \text{si } 0 = \alpha_j x_j + b_j \end{cases}
 \end{aligned}$$

Como se ve, el cálculo es por partes, combinando los gradientes de las partes diferenciables y haciendo un análisis cuidadoso de la función no diferenciable.

### 3.4.4. Subgradiente y esfuerzo computacional

Considere la siguiente función, que está definida en  $\mathbb{R}^n$ :

$$f(x) = x^T Q x + \|Ax\|_2^2 + \sum_{j=1}^n \max\{\alpha_j, x_j^4\}$$

donde  $Q \in \mathbb{R}^{n \times n}$  es simétrica y definida positiva,  $\alpha_j > 0, j = 1, \dots, n$ ,  $A \in \mathbb{R}^{n \times n}$ .

Esta función es evidentemente convexa y claramente no diferenciable. En este ejemplo vamos a determinar subgradientes y también analizaremos el esfuerzo computacional involucrado en el cálculo.

Para determinar  $\partial f(x)$  debemos notar que  $\sum_{j=1}^n \max\{\alpha_j, x_j^4\}$  no es diferenciable mientras que los demás términos si lo son.

- $\nabla x^T Q x = 2Qx$  debido a que  $Q$  es simétrica.
- $\nabla \|Ax\|_2^2 = 2A^T(Ax)$
- Definimos un subdiferencial  $h$  para  $\sum_{j=1}^n \max\{\alpha_j, x_j^4\}$  por lo que tenemos lo siguiente:

$$\nabla h(x) = \begin{cases} 4x_j^3 & \text{si } x_j < -\sqrt[4]{\alpha_j} \\ 4x_j^3 & \text{si } x_j > \sqrt[4]{\alpha_j} \\ 0 & \text{si } -\sqrt[4]{\alpha_j} < x_j < \sqrt[4]{\alpha_j} \\ \mu : \mu \in [4(-\sqrt[4]{\alpha_j})^3, 0] & \text{si } x_j = -\sqrt[4]{\alpha_j} \\ \nu : \nu \in [0, 4(\sqrt[4]{\alpha_j})^3] & \text{si } x_j = \sqrt[4]{\alpha_j} \end{cases}$$

Podemos también estimar una constante de Lipschitz restringida, asumiendo que sabemos que cualquier valor  $x$  que podamos encontrar en los algoritmos cumplirá con que  $\|x\|_2 \leq R$ . Entonces, mostraremos cómo estimar un valor para una constante de Lipschitz de  $f$  que sea válida en una región del espacio  $\mathbb{R}^n$  de a lo más radio  $R$  desde el origen, es decir, en  $B(0, R) = \{y : \|y\|_2 \leq R\}$ .

Para esto, lo más sencillo será descomponer por partes el gradiente y usar la propiedad de desigualdad triangular. Tomando  $\|\nabla f(x)\|_2 \leq L$  se tiene:

$$\nabla f(x) = 2Qx + 2A^T(Ax) + \begin{cases} 4x_j^3 & \text{si } x_j < -\sqrt[4]{\alpha_j} \\ 4x_j^3 & \text{si } x_j > \sqrt[4]{\alpha_j} \\ 0 & \text{si } -\sqrt[4]{\alpha_j} < x_j < \sqrt[4]{\alpha_j} \\ \mu : \mu \in [4(-\sqrt[4]{\alpha_j})^3, 0] & \text{si } x_j = -\sqrt[4]{\alpha_j} \\ \nu : \nu \in [0, 4(\sqrt[4]{\alpha_j})^3] & \text{si } x_j = \sqrt[4]{\alpha_j} \end{cases}$$

$$\nabla g(x) = 2Qx$$

$$\nabla p(x) = 2A^T(Ax)$$

$$\nabla h(x) = \begin{cases} 4x_j^3 & si & x_j < -\sqrt[4]{\alpha_j} \\ 4x_j^3 & si & x_j > \sqrt[4]{\alpha_j} \\ 0 & si & -\sqrt[4]{\alpha_j} < x_j < \sqrt[4]{\alpha_j} \\ \mu : \mu \in [4(-\sqrt[4]{\alpha_j})^3, 0] & si & x_j = -\sqrt[4]{\alpha_j} \\ \nu : \nu \in [0, 4(\sqrt[4]{\alpha_j})^3] & si & x_j = \sqrt[4]{\alpha_j} \end{cases}$$

Por lo tanto para cada gradiente tenemos:

$$\nabla g(x) \rightarrow \|2Qx\|_2 \leq 2\|Q\|_2 \cdot \|x\|_2 = 2\|Q\|_2 \cdot R$$

$$\nabla p(x) \rightarrow \|2A^T(Ax)\|_2 \leq 2\|A^T A\|_2 \cdot \|x\|_2 = 2\|A^T A\|_2 \cdot R$$

$$\nabla h(x) \rightarrow \sqrt{n} \cdot 4 \cdot R^3$$

Por lo tanto el L nos da:

$$L = 2\|Q\|_2 \cdot R + 2\|A^T A\|_2 \cdot R + \sqrt{n} \cdot 4 \cdot R^3$$

Ahora vamos a estimar el esfuerzo computacional, en flops, de calcular un subgradiente de  $f$ . Para estimar el esfuerzo computacional que se requiere para calcular el subgradiente de  $f$ , es necesario analizar cada una de las componentes por separado:

- $2Qx \rightarrow$  Es una matriz por un vector, así que por lo aprendido en el curso sabemos que el orden es  $O(n^2)$ .
- $2A^T(Ax) \rightarrow (Ax)$  es una matriz por un vector, que queda como un vector ( $\mu$ ), y luego tenemos  $A^T \mu$ , lo que es una matriz por vector de nuevo y nos da un orden final de  $O(n^2)$ . Alternativamente podemos tener una matriz  $A' = A^T A$  entonces  $A'x$  constituye a una matriz por vector que corresponde a un orden de  $O(n^2)$ .

■

$$\nabla h(x) = \begin{cases} 4x_j^3 & si & x_j < -\sqrt[4]{\alpha_j} \\ 4x_j^3 & si & x_j > \sqrt[4]{\alpha_j} \\ 0 & si & -\sqrt[4]{\alpha_j} < x_j < \sqrt[4]{\alpha_j} \\ \mu : \mu \in [4(-\sqrt[4]{\alpha_j})^3, 0] & si & x_j = -\sqrt[4]{\alpha_j} \\ \nu : \nu \in [0, 4(\sqrt[4]{\alpha_j})^3] & si & x_j = \sqrt[4]{\alpha_j} \end{cases} \quad \text{En el peor de los casos, } O(n^2)$$

Por lo tanto, se concluye que el esfuerzo computacional máximo, en flops, es de  $O(n^2)$ .

Entonces, si bien el Método del Subgradiente tiene una iteración sencilla, de la forma  $x^{k+1} = x^k - \lambda h$ , que es  $O(n)$ , la determinación de los subgradientes hace que el esfuerzo computacional neto por iteración sea  $O(n^2)$ .

### 3.4.5. Subgradiente y esfuerzo computacional 2

Consideremos la siguiente función en  $n$  variables:

$$f(x) = \|Ax - b\|_+ - \sum_{i=1}^m \log(b_i - \alpha_i^T x)$$

donde  $\|u\|_+ = \sum_{j=1}^n \max\{0, u_j\}$ , y  $A \in \mathbb{R}^{m \times n}$  y  $\alpha_1, \dots, \alpha_m$  son las filas de  $A$ .

Primero, analicemos el dominio y propiedades de esta función.

La función está formada por la suma de dos términos. El primero es:

$$\|Ax - b\|_+ = \sum_{i=1}^m \max\{0, \alpha_i^T x - b_i\}$$

Esta función es suma de los términos de la forma  $\max\{0, u_j\}$ , que son funciones convexas (basta verlo en un gráfico) y no decrecientes. La expresión anterior es la composición de estas con una función lineal, que es convexa. Luego, este primer término es una función convexa.

El segundo término es la suma de funciones que corresponden a la composición de  $-\log$ , que es creciente, y funciones lineales, que son convexas. Luego, eso también es convexo y la función  $f$  completa es convexa.

Respecto al dominio, notemos que el primer término está bien definido para cualquier  $x \in \mathbb{R}^n$ . Sin embargo, el segundo término, como involucra a la función logaritmo, está definido sólo si  $b_i - \alpha_i^T x > 0$ , que es lo mismo que  $\alpha_i^T < b_i$ , para todo  $i$ . Entonces, el dominio de la función  $f$  es el conjunto  $D = \{x \in \mathbb{R}^n : \alpha_i^T < b_i, i = 1, \dots, m\}$ , es decir, el interior del poliedro  $Ax \leq b$ .

Ahora vamos a determinar  $\partial f(x)$  en un punto cualesquiera del dominio de  $f$ . En forma extendida, la función es

$$f(x) = \|Ax - b\|_+ - \sum_{i=1}^m \log(b_i - \alpha_i^T x)$$

Sea  $g(u) = \|u\|_+$ . Entonces, tenemos:

$$f(x) = g(Ax - b) - \sum_{i=1}^m \log(b_i - \alpha_i^T x)$$

Entonces, para calcular el subdiferencial, hacemos (abusando, un poco, la notación, tal como lo hicimos en clases):

$$\partial f(x) = \partial g(Ax - b) - \partial\left(\sum_{i=1}^m \log(b_i - \alpha_i^T x)\right)$$

Para el segundo término, que es diferenciable, tenemos:

$$\frac{\partial}{\partial x_j} \left( \sum_{i=1}^m \log(b_i - \alpha_i^T x) \right) = - \sum_{i=1}^m \frac{\alpha_{ij}}{b_i - \alpha_i^T x}$$

Ahora, para el primer término, tenemos:

$$\partial g(Ax - b) = A^T \partial g(Ax - b)$$

Estudiemos el subdiferencial de  $g$ . Tenemos que  $g(u) = \sum_{i=1}^n \max\{0, u_i\}$  (notemos que aquí cambiamos el índice respecto a la definición original ya que esa definición es genérica y acá lo estamos aplicando explícitamente a las filas de  $Ax - b$ , que están indexadas por  $i$ , y son  $m$ ), así que

$$\partial g(u) = \sum_{i=1}^n \partial \max\{0, u_i\}$$

Esto se puede analizar de la misma forma que lo hicimos en clases para la  $\|x\|_1$  ya que es suma de términos que sólo dependen de las componentes individuales. Entonces, basta analizar cómo es la situación en una variable y luego extenderlo al vector completo. Esto lleva a :

$$p(u) \in \partial g(u) \Leftrightarrow p(u)_i = \begin{cases} 1 & u_i > 0 \\ 0 & u_i < 0 \\ [0, 1] & u_i = 0 \end{cases} \quad i = 1, \dots, m$$

Notemos que tenemos una composición de funciones y aquí nos va a ser útil la siguiente “regla de la cadena” para funciones no diferenciables: sea  $f$  una función convexa, si  $p \in \partial f(x)$  y  $g(x) = f(Bx)$ , donde  $B$  es alguna matriz de dimensiones adecuadas, entonces  $B^T p \in \partial g(x)$ . Luego, usando ese resultado, se tiene que un subgradiente de  $f$  tiene la forma

$$A^T p(Ax - b) + h(x)$$

donde  $h(x)$  está dado por la expresión de arriba de las derivadas parciales del segundo término.

Ahora estudiemos el esfuerzo computacional, en función de  $n$  y  $m$ , en el que se incurre para determinar un elemento de  $\partial f(x)$ , para un  $x$  dado en el dominio de  $f$ .

Para evaluar un subgradiente tenemos: para el primer término, el mayor esfuerzo es la multiplicación  $A^T p$ . Como  $A \in \mathbb{R}^{m \times n}$ , el esfuerzo es de  $O(mn)$ . Para el segundo término, cada deriva parcial depende de las filas de  $Ax - b$ . Esto debería evaluarse antes y requiere, también,  $O(mn)$  flop. Luego, para cada derivada parcial hay que hacer  $m$  divisiones, y eso es para cada  $j$ , así que es  $O(mn)$ . Luego, el esfuerzo total es  $O(mn)$ .

### 3.4.6. Método de Primer Orden proyectado

Considere el problema de optimización

$$\begin{aligned} \text{mín} \quad & \frac{1}{2} x^T Q x + c^T x \\ \text{s.a.} \quad & Ax = b \end{aligned}$$

Donde  $Q \in \mathbb{R}^{n \times n}$  es simétrica y definida positiva,  $c \in \mathbb{R}^n$ ,  $A \in \mathbb{R}^{m \times n}$ ,  $b \in \mathbb{R}^m$ , con  $m < n$ . Además, se asume que  $A$  tiene rango completo. Queremos aplicar el método simple de primer orden, pero en modo “proyectado” a este problema.

Tenemos que, si estamos en algún punto  $x^k$  de las iteraciones,  $\nabla f(x^k) = Qx + c$ . Este es el vector que necesitaremos proyectar, en cada iteración, sobre el conjunto definido por  $Ax = b$ . Entonces, primero abordamos el problema de proyección. Supongamos que nos dan un vector  $u$  cualquiera. La proyección se calcula resolviendo el siguiente problema:

$$\begin{aligned} \text{mín} \quad & \frac{1}{2} \|x - u\|_2^2 \\ \text{s.a.} \quad & Ax = b \end{aligned}$$

La solución de este problema es conocida. Hay que formar el lagrangiano del problema:

$$L(x, \lambda) = \frac{1}{2} \|x - u\|_2^2 + \lambda^T (Ax - b)$$

donde  $\lambda$  es el vector de  $m$  multiplicadores de Lagrange.

Derivando e igualando a cero se obtiene que  $x = u + A^T \lambda$  y ahora usamos la restricción  $Ax = b$  para despejar  $\lambda$ :

$$Ax = Au + AA^T \lambda$$

de donde se obtiene que  $\lambda = (AA^T)^{-1}(b - Au)$ . La matriz  $(AA^T)$  es invertible ya que  $A$  tiene rango máximo. Reemplazando en la expresión de  $x$  se obtiene, finalmente:

$$x = (I - A^T(AA^T)^{-1}A)u + A^T(AA^T)^{-1}b$$

Esto es la proyección de  $u$  y lo tenemos que aplicar al paso del algoritmo. Usando la notación  $\pi(u)$  para la proyección,

$$\begin{aligned} x^{k+1} &= \pi(x^k - \lambda_k \nabla f(x^k)) \\ &= (I - A^T(AA^T)^{-1}A)(x^k - \lambda_k \nabla f(x^k)) + A^T(AA^T)^{-1}b \\ &= (I - A^T(AA^T)^{-1}A)(x^k - \lambda_k(Qx^k + c)) + A^T(AA^T)^{-1}b \end{aligned}$$

Ahora vamos a estimar el orden de número de operaciones de punto flotante que se requieren por iteración y vamos a comparar esto con lo que habría que hacer si sólo que se quisiera resolver

$$\min \frac{1}{2} x^T Q x + c^T x$$

La fórmula desarrollada se ve compleja, pero hay que tener algo de cuidado: lo más costoso que ahí aparece es  $(AA^T)^{-1}$ , pero esto basta calcularlo *una sola vez* al comienzo del algoritmo, aunque es verdad que requiere  $O(n^3)$  flops, pero sólo una vez. En las iteraciones generales, hay que evaluar siempre  $Qx^k + c$  y eso es  $O(n^2)$ . Esto mismo es el esfuerzo adicional por iteración en el método sin restricciones. El método proyectado requiere, además hacer la multiplicación por la matriz de proyección, que es otro  $O(n^2)$ . En total, el método proyectado es  $O(n^2)$ , “igual” al método sin proyección. Pero aquí podemos caer víctimas de la notación  $O()$ ; en efecto, el método proyectado requiere, al menos el doble de esfuerzo que el no proyectado, y además el gastar  $O(n^3)$  una vez al comienzo. Si  $n$  es grande, eso es igual un esfuerzo enorme. En otras palabras, realmente es importante que la etapa de proyección sea muy barata.

### 3.4.7. Método no diferenciable proyectado

Vamos a considerar nuevamente un problema de estimación estadística. Tenemos  $m$  observaciones de un cierto fenómeno,  $b_1, \dots, b_m$  y queremos explicarlo en función de  $n$  factores, para los cuales también existen valores conocidos  $a_{ij}$ , que es el valor del factor  $j$  para la observación  $i$ . Entonces, se postula un modelo lineal de la forma:

$$\sum_{j=1}^n a_{ij} x_j = b_i, \quad i = 1, \dots, m$$

en donde los  $x_j$  son los pesos de los factores. Entonces, queremos encontrar la mejor solución posible al sistema de ecuaciones  $Ax = b$ . Este problema lo hemos planteado ya en el curso, usando mínimos cuadrados pero acá usaremos un criterio distinto, el error en valor absoluto. Adicionalmente, por propiedades del problema subyacente, se sabe que los coeficientes de la estimación, las variables  $x$ , deben estar dentro de ciertos rangos específicos y conocidos. Entonces, se plantea resolver el siguiente problema de optimización:

$$\begin{aligned} \min \quad & \|Ax - b\|_1 \\ \text{s.a.} \quad & l_j \leq x_j \leq u_j, j = 1, \dots, n \end{aligned}$$

donde  $A \in \mathbb{R}^{m \times n}$ ,  $b \in \mathbb{R}^m$  y los  $l_j, u_j$  son todos escalares conocidos.

Como usted debe observar, la función objetivo de este problema es no diferenciable, así que vamos a determinar un subgradiente de la función objetivo del problema en un punto  $x$  factible. Sea  $f(x) = \|Ax - b\|_1 = \sum_{i=1}^m |\alpha_i^T x - b_i|$

Luego, se deriva usando la regla de la cadena no diferenciable, que ya se mencionó en el ejemplo 3.4.5:

$$\partial f(x) = \partial \|Ax - b\|_1 = A^T \partial \|Ax - b\|_1 = A^T p(x)$$

Con  $p(x)$  definido por

$$p_i(x) = \begin{cases} 1 & \text{si } \alpha_i^T x - b_i > 0 \\ -1 & \text{si } \alpha_i^T x - b_i < 0 \\ u \in [-1, 1] & \text{si } \alpha_i^T x - b_i = 0 \end{cases}$$

Ahora, si queremos usar el Método del Subgradiente, tenemos que el problema tiene restricciones, pero estas son simples. Entonces, se debe proyectar  $x$  en cada iteración del algoritmo, sobre el conjunto  $D = \{x : \forall j \, l_j \leq x_j \leq u_j\}$ .

Lo que se debe hacer es revisar cada coordenada  $x_j$  y determinar si es que está se encuentra dentro del conjunto  $D$ . Si  $l_j \leq x_j \leq u_j$ , entonces esta coordenada queda igual. Por otro lado, si  $x_j < l_j$ , esta coordenada se debe poner en  $l_j$  y si  $x_j > u_j$ , se debe llevar a  $u_j$ . Esto es:

$$\Pi_D(x)_j = \begin{cases} x_j & \text{si } l_j \leq x_j \leq u_j \\ l_j & \text{si } x_j < l_j \\ u_j & \text{si } x_j > u_j \end{cases}$$

Ahora le proponen hacer una transformación al problema y poder resolverlo con algún algoritmo, ojalá, más eficiente que el método de subgradiente. Para esto, considere la función  $g : \mathbb{R} \rightarrow \mathbb{R}$  dada por  $g(u) = \sqrt{u^2 + \mu}$  donde  $\mu > 0$  es un parámetro dado. Vamos a mostrar que  $g(u) \approx |u|$ , es decir, se puede usar para aproximar el valor absoluto, y que esa aproximación se hace cada vez mejor mientras más chico sea  $\mu$ .

Se tiene que

$$g(u) = \sqrt{u^2 + \mu} = \sqrt{u^2(1 + \frac{\mu}{u^2})} = |u|\sqrt{1 + \frac{\mu}{u^2}}$$

Para valores de  $u$  muy grandes,  $\frac{\mu}{u^2} \rightarrow 0$ , por lo que  $\sqrt{1 + \frac{\mu}{u^2}} \approx \sqrt{1} = 1$ . Luego

$$g(u) \approx |u| \cdot 1 = |u|$$

Por otro lado, para valores de  $u$  más pequeños, la aproximación no sería tan buena. Ahora bien, si  $\mu \rightarrow 0 \Rightarrow \frac{\mu}{u^2} \rightarrow 0$ , por lo que a través del mismo razonamiento anterior se concluye que  $g(u) \approx |u|$ .

Entonces, dado lo anterior, se propone ahora resolver, alternativamente, el problema:

$$\begin{aligned} \text{mín} \quad & \sum_{i=1}^m g(\alpha_i^T x - b_i) \\ \text{s.a.} \quad & l_j \leq x_j \leq u_j, j = 1, \dots, n \end{aligned}$$

donde  $\alpha_1, \dots, \alpha_m$  son las filas de la matriz  $A$ , usando un algoritmo basado en el Método Acelerado de Nesterov (no es necesario recordar todo el detalle). Esto lo podemos hacer pero se debe usar el Método Acelerado de Nesterov Proyectado, ya que el problema incluye restricciones acotadas en  $x$ , lo que requiere proyectar las soluciones propuestas en cada iteración sobre el conjunto factible  $[l, u]$ . Esta etapa se incorpora en el algoritmo y se maneja de la misma manera que se menciona anteriormente.

Cabe mencionar que las hipótesis necesarias se cumplen, efectivamente, la función objetivo, dada por

$$f(x) = \sum_{i=1}^m g(\alpha_i^T x - b_i)$$

es convexa y dos veces continuamente diferenciable.

Adicionalmente, dado que la solución se busca en un conjunto cerrado y acotado, se sabe que la norma del Hessiano de  $f$  (que es continua en  $x$ ) alcanzará su máximo dentro de este, por lo que es posible acotarlo y de ahí deducir que  $\nabla f$  cumple una propiedad de Lipschitz en el conjunto  $D$ , que es todo lo que se necesita.